

UNIVERSIDAD PERUANA UNIÓN
Facultad de Ingeniería y Arquitectura
Escuela Profesional de Ingeniería de Sistemas — Filial Juliaca

PROYECTO DE TESIS / PROYECTO INTEGRADOR

HemoScan Andino

Sistema no invasivo de tamizaje de anemia infantil con fusión óptica
multisensor y autocalibración por altitud

**Entregable 2 de Software (CE022, Parte 1) — Plataforma de
Datos del Sistema**

Juliaca, Puno, Perú
Julio de 2026

Índice

Resumen Ejecutivo	4
1. Modelo de Datos	5
1.1 Alcance de esta Parte 1 y relación con el Entregable N.º 5	6
1.2 Arquitectura de la información: cuatro dominios y su justificación	6
1.3 Modelo Entidad-Relación (nivel conceptual)	9
1.4 Reconciliación con el Diagrama de Clases del Entregable N.º 5	11
1.5 Modelo lógico relacional	13
1.6 Modelo normalizado: prueba de Primera, Segunda y Tercera Forma Normal	16
1.7 Diccionario de datos completo	18
Esquema gestion	18
Esquema geo	20
Esquema config	22
Esquema clinico	22
1.8 Síntesis de integridad referencial y de rendimiento	29
2. Implementación de Base de Datos	29
2.1 Entorno y metodología de validación	29
2.2 Script DDL: estructura y fragmentos representativos	30
2.3 Evidencia real de creación de tablas, relaciones y restricciones	32
2.4 Script DML: carga de datos ilustrativos	34
2.5 Evidencia real de consultas ejecutadas	36
2.6 Evidencia real de pruebas de integridad referencial y de dominio	38
2.7 Nota de alcance de esta implementación	40
3. Consultas y Programación en Base de Datos	40
4. Seguridad y Administración de Base de Datos	41
Anexos	42
Evidencia externa y reproducibilidad de la ejecución real	42
Anexo A — Script DDL: estructura completa y extracto representativo (ejecutado sin errores contra PostgreSQL 16.4 + PostGIS 3.4.3; script íntegro en Anexos-Ejecutables-Entregable-0)	
Anexo B — Script DML: estructura completa y extracto representativo (carga de datos ilustrativos, ejecutado sin errores; script íntegro en Anexos-Ejecutables-Entregable-06/02_dml_hemo checksum SHA-256 65e7f79bd0822260d3ed2cef90a6b5f0c0b261e01610dcfbfab0ae724fecc46b2)	48
Anexo C — Script completo de consultas de verificación (10 consultas, todas ejecutadas; idéntico a Anexos-Ejecutables-Entregable-06/03_consultas_verificacion.sql , checksum SHA-256 5fbf99528f0dd78b782ca0e42354fd0cd5d79b3d1a4b0e790a02873e19dc2458; salida cruda completa de las 10 consultas en log_03_consultas_ejecucion.txt)	51
Anexo D — Script completo de pruebas de integridad (7 pruebas de violación + 1 control positivo = 8 pruebas, todas con el resultado esperado; idéntico a Anexos-Ejecutables-Entregable-06/04_pruebas_integridad.sql , checksum SHA-256 b8b8a826e6083e988142db19b99df5fca92d862f9fb40f9fc452c61f2cac0b82; salida cruda completa con el detalle DETAIL : de cada error en log_04_pruebas_ejecucion.txt)	54
Anexo E — Glosario de términos técnicos de este entregable	55
Anexo F — Índice de diagramas de este entregable	56
Anexo G — Trazabilidad hacia el Entregable N.º 5	56
Autoevaluación frente a la rúbrica	57

Entregable N.º 6 — Plataforma de Datos del Sistema ### Parte 1 (Semestre 1):
Modelo de Datos e Implementación de Base de Datos

(Sexto entregable consecutivo del proyecto HemoScan Andino. Corresponde al Entregable N.º 2 de la competencia CE022 — Área C: Ingeniería de Software; utiliza una numeración de Entregable propia y distinta de la de los Entregables N.º 1-4 del Área B — Gestión de TI — y N.º 5 del Área C — Ingeniería de Software — ya presentados, aunque continúa la secuencia documental global del proyecto. Tal como lo anticipó explícitamente el Entregable N.º 7 — Diseño de Red, Área A — Infraestructura Tecnológica [“El número 6 se reserva a un entregable en elaboración paralela de otra área de competencia del mismo proyecto, no desarrollado en este documento”], este es precisamente ese entregable.)

Área de competencia: Área C — Ingeniería de Software **Competencia:** CE022 — Ingeniería de la Información **Evidencia que cubre este documento (Parte 1 de 2):** CE0221 (Modelo de datos relacional documentado y validado) y CE0222 (Base de datos relacional implementada y consultable). Las evidencias CE0223 (motor transaccional y procedimientos avanzados) y CE0224 (administración y seguridad de base de datos empresarial) corresponden a la **Parte 2**, de desarrollo previsto en el Semestre 2 (ver Secciones 3 y 4). **Paquete de la Estructura de Desglose del Trabajo (EDT) que satisface:** 4.2 — “Modelo de datos y mapeo FHIR” (Entregable N.º 3, sección 3.2.3), responsable Integrante 3, cuyo criterio de aceptación formal es “cada campo clínico mapeado a un recurso/atributo FHIR — atiende A5, A14 (Entregable N.º 4)”. Se distingue explícitamente del paquete EDT 2.2 — “Servidor/nube y base de datos” (Integrante 1, Entregable N.º 3), responsable del aprovisionamiento del **servidor físico/nube real** de la posta donde este modelo se desplegará en producción: este documento diseña e implementa (en un entorno de validación) el esquema; el paquete 2.2 aprovisiona el motor donde ese esquema correrá en Fase 0 real. **Sistema documentado:** HemoScan Andino — capa de datos relacional (PostgreSQL 16 + PostGIS 3.4) que sustenta el backend de interoperabilidad FHIR (FastAPI/NestJS), la aplicación móvil Flutter/PWA offline-first y la plataforma web de analítica geoespacial (React/TypeScript)

Organización diagnosticada (caso ilustrativo y representativo): Microred de Salud Altiplano-Puno

Integrantes del equipo:

N.º	Integrante	Rol / Área de competencia
1	<i>(Integrante 1, Líder de Infraestructura Tecnológica – nombre por completar)</i>	CE03 — Infraestructura Tecnológica (red, seguridad, centro de datos)
2	<i>(Integrante 2, Líder de Gestión de TI – nombre por completar)</i>	CE01 — Gestión de TI (diagnóstico, business case, plan de gestión, procesos)
3	<i>(Integrante 3, Líder de Ingeniería de Software – nombre por completar)</i>	CE02 — Ingeniería de Software (requerimientos, datos, sistema, calidad) — responsable principal de este entregable

Docente asesor(a) del curso: *(nombre del docente asesor por completar)* **Curso / Sílabo:** *(nombre del curso / sílabo por completar)* **Ciclo académico:** 2026-I (supuesto, a confirmar con la programación académica vigente)

Lugar y fecha: Juliaca, Puno, Perú — 07 de julio de 2026

Nota metodológica y alcance (léase antes de continuar). Este documento es el Entregable N.º 6 del proyecto HemoScan Andino, competencia **CE022 — Ingeniería de la Información**, y desarrolla **únicamente la Parte 1** del producto exigido por su rúbrica: la Sección 1 (Modelo de Datos) y la Sección 2 (Implementación de Base de Datos) se desarrollan en profundidad; las Secciones 3 (Consultas y Programación en Base de Datos) y 4 (Seguridad y Administración) se dejan como **placeholders de alcance explícito para una Parte 2, de desarrollo previsto en el Semestre 2**, siguiendo instrucción directa del encargo de este entregable. Se declaran seis límites de honestidad técnica y decisiones metodológicas, en la misma disciplina de transparencia ya sostenida en los cinco entregables anteriores del proyecto:

Primero, este modelo de datos **toma como insumo fijo y no reinterpretable** los identificadores, el diagrama de clases (Diagrama 6) y el mapeo a los seis recursos FHIR (sección 4.5) ya establecidos en el **Entregable N.º 5** (Requerimientos y Diseño del Sistema): las 40 RF, 20 RNF, 12 RN, 11 CU y 10 ADR allí fijados se citan tal cual, sin reenumerar. La sección 1.4 de este documento hace explícita, entidad por entidad, la trazabilidad y las diferencias de traducción entre aquel diagrama de clases (orientado a objetos de software) y este modelo relacional (orientado a persistencia normalizada).

Segundo, y de forma específica para este entregable, se incorporan **tres entidades nuevas** no presentes como clases en el Diagrama 6 del Entregable N.º 5 (**agente_comunitario**, la tabla técnica **recurso_fhir** y la tabla de unión **paciente_cuidador**), exigidas por el alcance explícito de la competencia CE022 (“entidades de gestión: usuarios/roles, microrredes, agentes comunitarios”) o por necesidades de normalización relacional que no existen a nivel de un diagrama de clases de software. Cada una se documenta explícitamente como una **extensión, nunca una contradicción**, del modelo previo. De igual manera, la entidad **establecimiento_salud** (equivalente funcional a un recurso FHIR *Location*) se incorpora como tabla de apoyo geoespacial-administrativo; **no** se cuenta como un séptimo recurso FHIR clínico y **no** contradice el ADR-06 del Entregable N.º 5 (que fijó el alcance en 6 recursos clínicos para Fase 0) — la aclaración completa se desarrolla en la sección 1.4.

Tercero, todos los valores clínicos exactos que dependen del texto íntegro de la **NTS 213-MINSA/DGIESP-2024** (puntos de corte de hemoglobina, fórmula exacta de corrección por altitud, posología de hierro) permanecen **[DATO PENDIENTE DE VALIDAR]**, tal como ya se declaró en los Entregables N.º 4 y N.º 5; el modelo de datos se diseña deliberadamente para parametrizar y versionar estos valores (tabla **config.configuracion_clinica**, JSONB versionado) en lugar de fijarlos, materializando la regla de negocio RN-02.

Cuarto, y en un esfuerzo deliberado por superar la limitación de “diseño no ejecutado” señalada como pendiente en entregables previos, **los scripts DDL y DML de este documento se ejecutaron realmente** contra una instancia efímera de PostgreSQL 16.4 + PostGIS 3.4.3, desplegada en un contenedor Docker local (imagen **postgis/postgis:16-3.4**, digest inmutable **sha256:44126d872ac91993766c341e369c539e819661432176**) únicamente para fines de validación de este entregable, el 2026-07-07. A diferencia de las cinco entregas anteriores del proyecto, esta afirmación **no depende únicamente de la palabra de este documento**: los cuatro scripts (DDL, DML, consultas de verificación y pruebas de integridad), el **docker-compose.yml** exacto usado y las transcripciones crudas y sin editar de cada corrida quedan archivados co-

mo **archivos separados, verificables de forma independiente por checksum SHA-256**, en la carpeta hermana **Anexos-Ejecutables-Entregable-06/** (ver su `README.md` para el detalle de checksums, el procedimiento exacto de reproducción y las dos cifras que la propia ejecución real permitió corregir frente a lo que se había narrado). La Sección 2 y los Anexos A a D de este documento resumen y citan esa evidencia; el registro crudo completo vive únicamente en esos archivos externos, no solo como texto narrativo dentro de este Markdown. Se declara con la misma honestidad que **esta instancia efímera no es, bajo ninguna interpretación, el servidor de producción de la posta**: ese aprovisionamiento sigue siendo responsabilidad del paquete EDT 2.2 del Integrante 1, como ya se aclaró en el encabezado de este documento. El contenedor efímero ya fue destruido tras capturar la evidencia (`docker compose down`); lo que persiste y es reproducible en cualquier momento son los scripts, el `docker-compose.yml` y los logs crudos, no el contenedor en sí.

Quinto, todos los datos cargados mediante el script DML (Sección 2.4, Anexo B) son **enteramente ficticios e ilustrativos**, generados únicamente para validar que el esquema funciona con datos de forma realista. **Ningún dato de este documento corresponde a una niña, niño, cuidador/a o profesional de salud real**: no existe todavía convenio institucional con ninguna microrred (Nota metodológica del Entregable N.º 5), por lo que —dada además la máxima sensibilidad de tratarse de datos de salud de menores de edad bajo la Ley N.º 29733— construir o utilizar un dataset real está fuera de alcance ético y de posibilidad práctica de este entregable.

Sexto, sobre la extensión de este documento frente al límite de 15-25 páginas de la plantilla del producto: esta Parte 1 desarrolla en profundidad únicamente 2 de las 4 secciones exigidas por la rúbrica completa (Modelo de Datos e Implementación de Base de Datos; las Secciones 3 y 4 son *placeholders* de alcance para el Semestre 2), y además incorpora, como evidencia directa de ejecución real, cuatro anexos (A-D) con scripts y salidas de motor que un documento puramente de diseño no necesitaría. Aun así, se reconoce con la misma honestidad de esta nota que el documento, en su versión final, sigue siendo más extenso de lo que ese límite sugiere: se condensaron los Anexos A y B (que antes reproducían íntegros los 429+348 líneas de los scripts DDL y DML dentro del propio Markdown) a un extracto representativo, moviendo el contenido completo y verificable a los archivos externos de **Anexos-Ejecutables-Entregable-06/**, lo que evita duplicar ese contenido dos veces dentro de este documento y reduce su extensión; no se afirma, sin embargo, que el resultado final quede ya dentro del rango de 15-25 páginas de un producto completo — eso corresponde evaluarlo formalmente contra la plantilla en la integración final con la Parte 2 (Semestre 2), cuando el documento combinado exista y pueda paginarse como una sola entrega.

Resumen Ejecutivo

Si HemoScan Andino es, en esencia, un sistema que transforma tres señales ópticas de bajo costo en una decisión de salud pública auditable, entonces **la plataforma de datos es el componente que hace posible que esa transformación sea confiable, trazable y reproducible en el tiempo** — no un contenedor pasivo de registros, sino el mecanismo que materializa, a nivel de motor de base de datos y no solo de intención de diseño, tres compromisos ya asumidos por el proyecto en entregables anteriores: que ninguna determinación de hemoglobina pierda su cadena de corrección por altitud (RN-02, RN-03), que ninguna captura de datos de un menor

de edad ocurra sin un consentimiento vigente verificable (RN-04) y que ningún acceso a datos clínicos quede sin registro auditable (RN-11). Este Entregable N.º 6, competencia **CE022 — Ingeniería de la Información**, desarrolla la primera mitad de esa plataforma: el **modelo de datos** (conceptual, lógico y normalizado, con su diccionario de datos completo) y su **implementación real** en PostgreSQL 16 con la extensión PostGIS, dejando para una Parte 2 en Semestre 2 la programación avanzada de base de datos (funciones, disparadores, procedimientos almacenados) y la administración de seguridad a nivel de motor (roles de base de datos, cifrado de columna, políticas de auditoría del propio gestor).

El modelo resultante organiza **24 tablas y 1 vista derivada** en **cuatro esquemas** que reproducen, a nivel de base de datos, la misma modularidad de dominio ya declarada arquitectónicamente en el ADR-02 del Entregable N.º 5 (monolito modular): **clinico** (15 tablas alineadas a los recursos FHIR Patient, Consent, Device, Observation, Provenance y AuditEvent), **geo** (3 tablas geoespaciales sobre PostGIS: microneces, establecimientos de salud con altitud y la capa agregada de hexágonos H3), **config** (2 tablas de parámetros clínicos versionados, que materializan la regla de negocio RN-02) y **gestion** (4 tablas de usuarios, roles RBAC, agentes comunitarios de salud y alertas de stock). El diseño resuelve explícitamente dos problemas de modelado no triviales heredados del carácter FHIR del proyecto: la referencia polimórfica del recurso **AuditEvent** (que en el estándar FHIR puede apuntar a cualquiera de otros cinco tipos de recurso) se resuelve mediante una tabla técnica compartida (**recurso_fhir**) que preserva integridad referencial real de PostgreSQL, evitando una clave foránea débil validada solo en aplicación; y la cardinalidad 1:1 de la cadena Captura -> CorrecciónAltitud -> Determinación (ya definida en el Diagrama 6 del Entregable N.º 5) se refuerza en el esquema físico mediante restricciones **UNIQUE**, en lugar de colapsarse en una única tabla ancha, preservando así a nivel de base de datos el mismo principio de independencia auditable que el ADR-05 exige a nivel de componentes de software.

De manera consistente con la disciplina de honestidad técnica del proyecto, este entregable no se limita a presentar scripts SQL redactados en papel: **el script DDL completo (24 tablas, 1 vista, 40 restricciones CHECK, 39 llaves foráneas, 14 restricciones UNIQUE y 60 índices) y el script DML de carga inicial se ejecutaron realmente** contra una instancia de PostgreSQL 16.4 con PostGIS 3.4.3 desplegada en un contenedor Docker local, y su salida real —incluyendo consultas de verificación (uniones múltiples, agregaciones, consultas geoespaciales con **ST_Distance** y **ORDER BY <->**, y una traza de auditoría completa) y siete pruebas deliberadas de violación de restricciones más un control positivo, todas con el resultado correcto— se documenta en la Sección 2 y en los Anexos A a D. A diferencia de una simple afirmación narrativa, esta ejecución quedó archivada como evidencia externa al propio documento —cuatro scripts **.sql**, el **docker-compose.yml** usado y las transcripciones crudas de cada corrida, todo verificable por checksum SHA-256 en la carpeta **Anexos-Ejecutables-Entregable-06/—**, de modo que la afirmación “se ejecutó realmente” no depende únicamente de la palabra de este Markdown. Esta evidencia de ejecución real e independientemente verificable, y no solo de diseño en papel, es la principal fortaleza que este entregable aporta frente a los cinco documentos anteriores del proyecto, todos ellos —por diseño y por etapa— ejercicios de gabinete no ejecutados.

1. Modelo de Datos

1.1 Alcance de esta Parte 1 y relación con el Entregable N.º 5

El Entregable N.º 5 ya produjo, en su sección 4.2 (Diagrama 6) y su sección 4.5, un diagrama de clases de 20 entidades y un mapeo detallado a los seis recursos FHIR del proyecto. Ese trabajo fue deliberadamente un ejercicio de **diseño orientado a objetos de software** (clases, atributos, asociaciones), no un modelo relacional normalizado ni un esquema físico de base de datos: el propio Entregable N.º 5 lo declara así en su alcance (sección 4, “Diseño Detallado”). La tarea específica de la competencia **CE022 — Ingeniería de la Información** es distinta y complementaria: tomar ese diseño como insumo fijo y producir a partir de él (a) un modelo entidad-relación explícito, (b) su forma lógica relacional, (c) la prueba de que dicha forma cumple las reglas de normalización, (d) un diccionario de datos exhaustivo y (e) su implementación real y consultable en un motor de base de datos concreto — PostgreSQL 16 con la extensión PostGIS, ya fijado como decisión arquitectónica en el ADR-10 del Entregable N.º 5.

Esta Parte 1 cubre íntegramente las evidencias **CE0221** (modelo de datos relacional documentado y validado) y **CE0222** (base de datos relacional implementada y consultable). Quedan fuera de alcance de esta Parte 1 — y se marcan explícitamente como tales en las Secciones 3 y 4 — las evidencias **CE0223** (funciones, disparadores y procedimientos almacenados; control transaccional avanzado) y **CE0224** (roles de base de datos distintos de los de aplicación, cifrado a nivel de columna, políticas de auditoría del motor, replicación y recuperación ante desastres), cuyo desarrollo está previsto para una Parte 2 en el Semestre 2, siguiendo instrucción explícita del encargo de este entregable.

1.2 Arquitectura de la información: cuatro dominios y su justificación

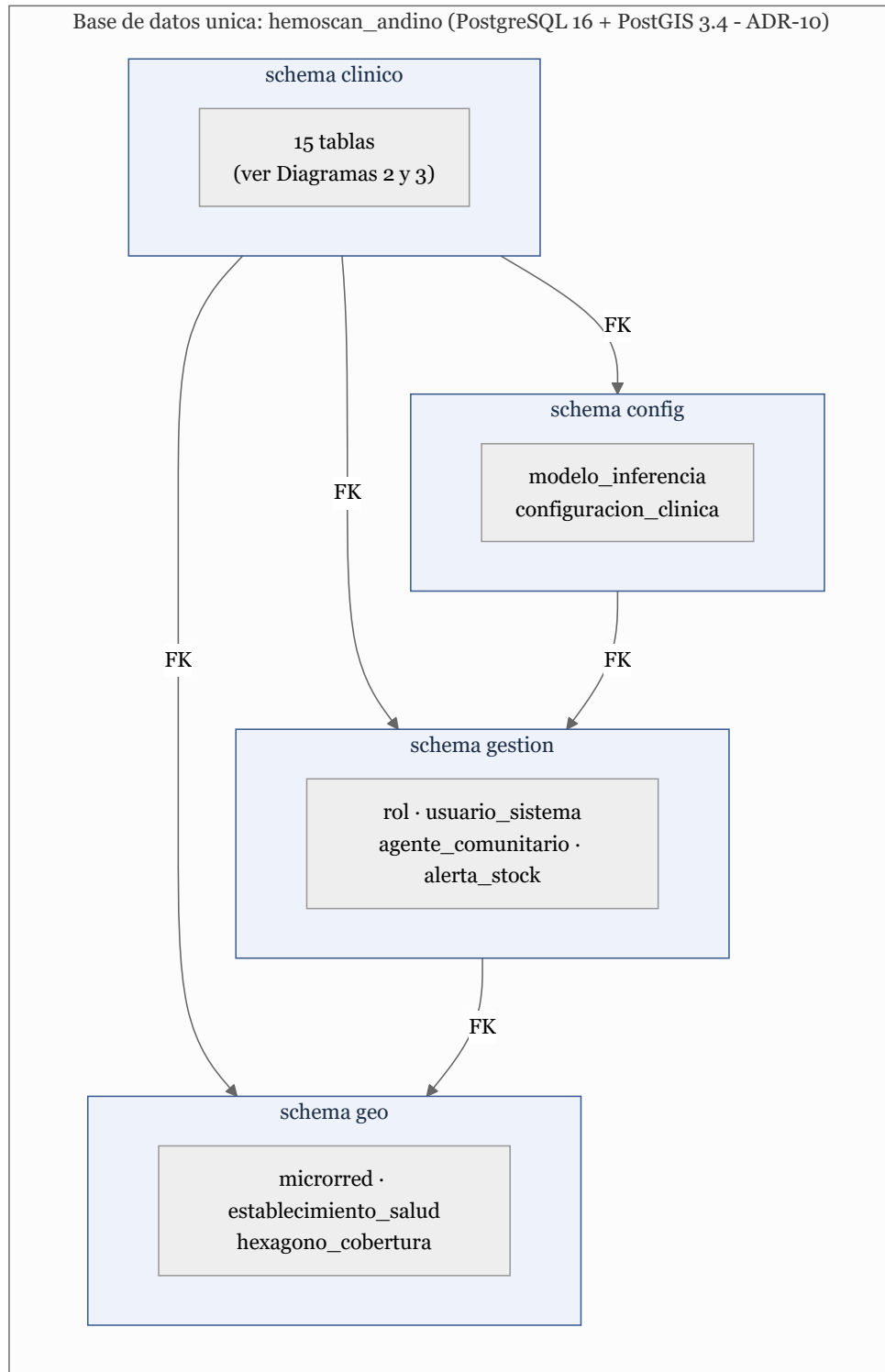
El modelo se organiza en cuatro esquemas (*namespaces* de PostgreSQL) que agrupan las tablas por dominio de responsabilidad, replicando a nivel de base de datos la misma frontera de módulos que el ADR-02 del Entregable N.º 5 ya fijó a nivel de componentes de backend (identidad, clínico/FHIR, geoespacial, etc.), sin que esto implique, en ningún caso, una arquitectura de microservicios ni bases de datos físicamente separadas — los cuatro esquemas conviven en una única base de datos (**hemoscan_andino**), consistente con el ADR-10 (PostgreSQL/PostGIS como único motor).

Esquema	Propósito	N.º de tablas	Tablas
clinico	Entidades clínicas alineadas a los seis recursos FHIR del proyecto (Patient, Consent, Device, Observation, Provenance, AuditEvent) y sus entidades de apoyo directo	15	recurso_fhir, paciente, cuidador, paciente_cuidador, consentimiento, dispositivo, captura, correccion_altitud, determinacion, alerta_derivacion, caso_confirmacion, prescripcion, provenance, auditevent, control_mensual
geo	Organización territorial y agregación geoespacial sobre PostGIS	3	microrred, establecimiento_salud, hexagono_cobertura
config	Parámetros clínicos versionados y auditables (materializa RN-02)	2	modelo_inferencia, configuracion_clinica

Esquema	Propósito	N.º de tablas	Tablas
<code>gestion</code>	Usuarios, roles RBAC, agentes comunitarios de salud y alertas administrativas	4	<code>rol</code> , <code>usuario_sistema</code> , <code>agente_comunitario</code> , <code>alerta_stock</code>

Convención de nomenclatura. Todos los identificadores técnicos (esquemas, tablas, columnas, restricciones) se escriben en **snake_case**, en español, **sin tildes ni ñes** (p. ej. `senal_ppg_ref`, no `señal_ppg_ref`), siguiendo una práctica estándar de portabilidad de bases de datos que evita ambigüedades de codificación de caracteres entre entornos (Windows/Linux, distintas configuraciones de *locale*). Esta convención afecta únicamente a los nombres técnicos internos; los textos visibles al usuario final (interfaz de la app y de la plataforma web) sí conservan la ortografía correcta del español, por no ser competencia de este entregable.

Diagrama 1 — Vista general de la arquitectura de información (cuatro esquemas y sus dependencias).



1.3 Modelo Entidad-Relación (nivel conceptual)

Se presentan cinco diagramas complementarios en notación entidad-relación (*crow's foot*), especificados como diagrama-como-código y renderizados a imagen vectorial con un renderizador libre de licencia (Mermaid); las multiplicidades se leen 1, 0..1, 0..*, 1..* sobre cada relación. Por legibilidad, dado que el esquema *clinico* concentra 15 tablas, se separa su presentación en dos diagramas: el núcleo de la cadena clínica (Diagrama 2) y la resolución de la referencia polimórfica de auditoría (Diagrama 3).

Diagrama 2 — Modelo E-R del núcleo clínico (esquema clinico, cadena de valor principal).

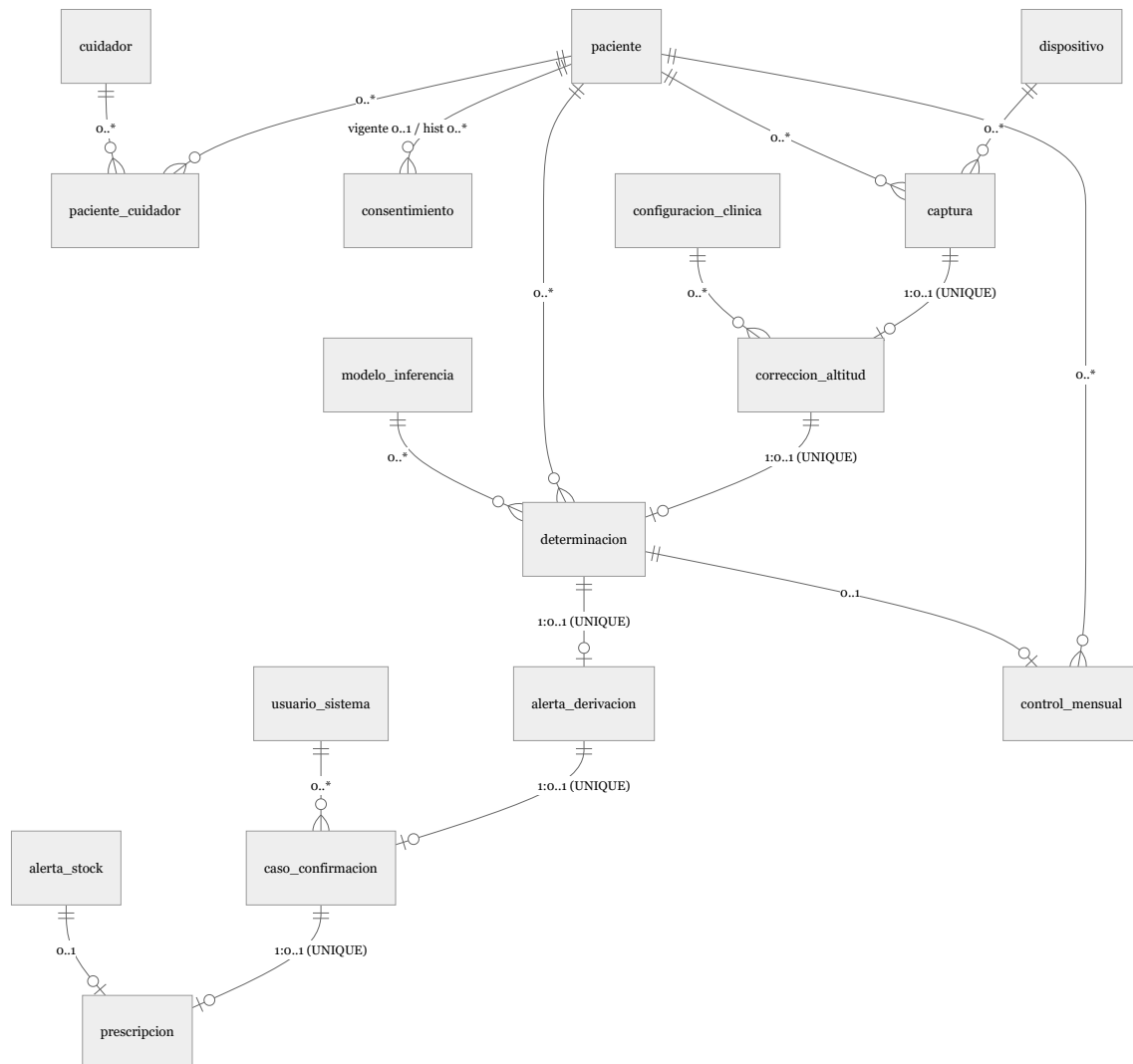


Diagrama 3 — Resolución de la referencia polimórfica FHIR y trazabilidad de auditoría.

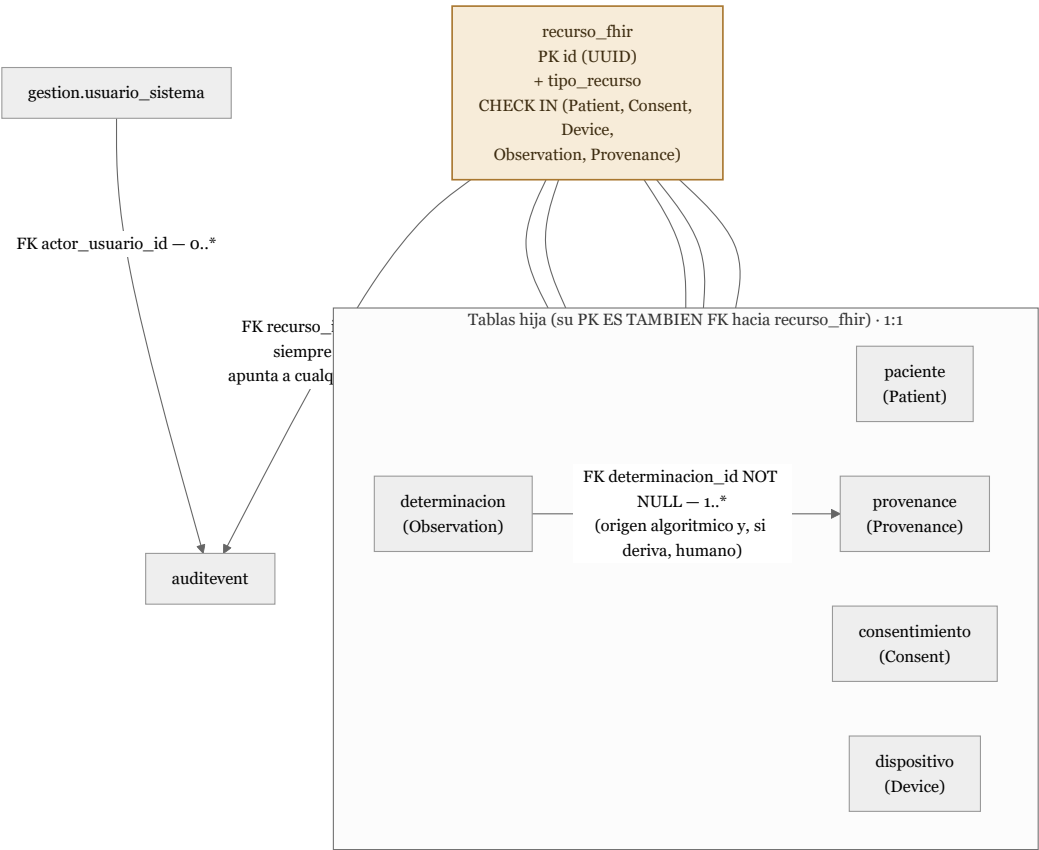


Diagrama 4 — Modelo E-R del dominio geoespacial (esquema geo, PostGIS).

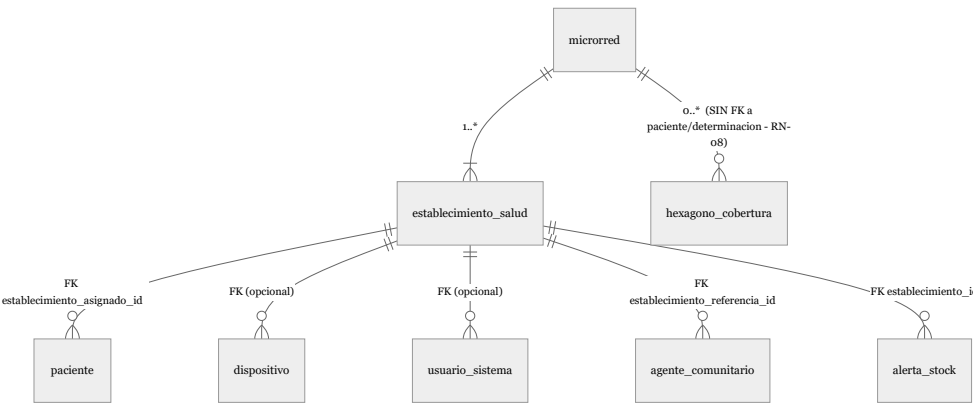
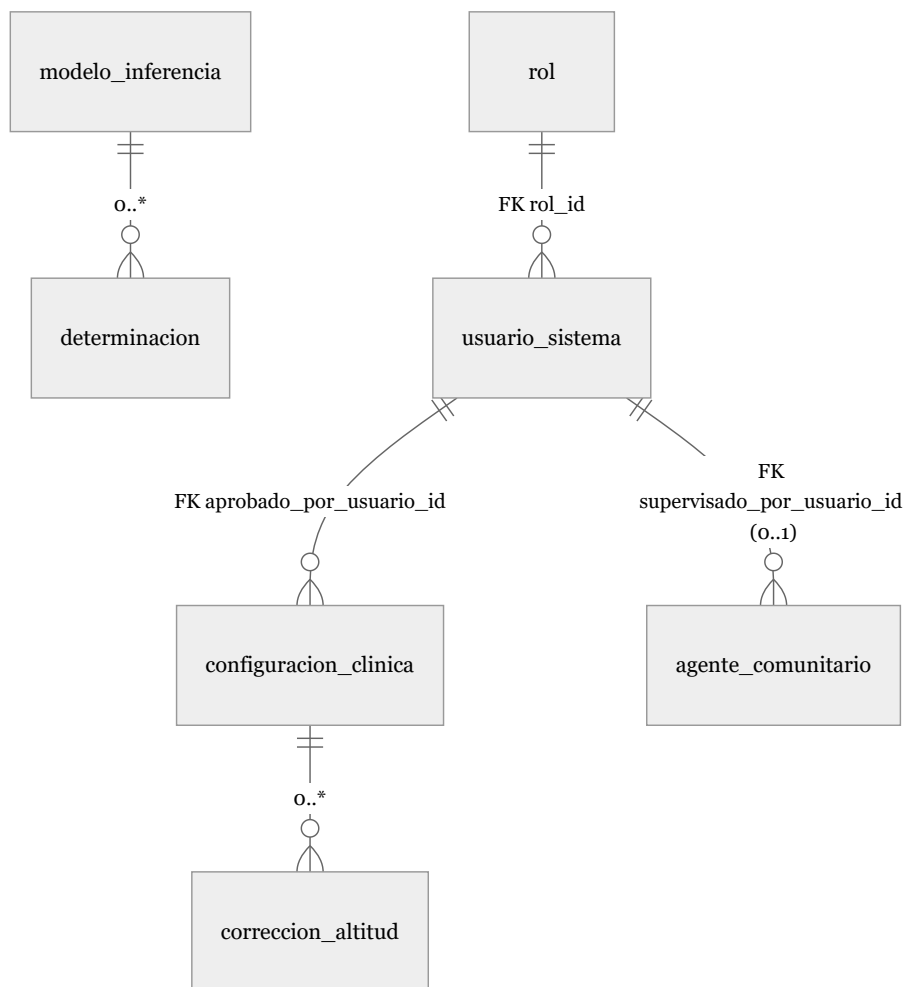


Diagrama 5 — Modelo E-R del dominio de configuración clínica y de gestión (esquemas config y gestion).



1.4 Reconciliación con el Diagrama de Clases del Entregable N.º 5

Esta sección es, deliberadamente, la más importante desde el punto de vista de la coherencia entre entregables: demuestra, clase por clase, que el modelo relacional de este documento **no diverge arbitrariamente** del diagrama de clases ya validado, sino que lo traduce con reglas explícitas y documentadas.

Clase del Diagrama 6 (Entregable N.º 5)	Tabla(s) de este modelo	Tipo de traducción
Paciente	clinico.paciente (+ clinico.recurso_fhir)	Directa, con adición de nombres/apellidos (ver nota A)
Cuidador	clinico.cuidador	Directa, con reubicación de parentesco (ver nota B)
Consentimiento (Consent)	clinico.consentimiento (+ clinico.recurso_fhir)	Directa
Dispositivo (Device)	clinico.dispositivo (+ clinico.recurso_fhir)	Directa
Captura	clinico.captura	Directa
CorreccionAltitud	clinico.correccion_altitud	Directa
Determinacion (Observation)	clinico.determinacion (+ clinico.recurso_fhir)	Directa
AlertaDerivacion	clinico.alerta_derivacion	Directa

Clase del Diagrama 6 (Entregable N.º 5)	Tabla(s) de este modelo	Tipo de traducción
CasoConfirmacion	clinico.caso_confirmacion	Directa
Prescripcion	clinico.prescripcion	Directa
AuditEvent	clinico.auditevent	Directa, con FK real hacia recurso_fhir (ver nota C)
Provenance	clinico.provenance (+ clinico.recurso_fhir)	Directa, con cardinalidad reconciliada (ver nota D)
ModeloInferencia	config.modelo_inferencia	Directa
ConfiguracionClinica	config.configuracion_clinica	Directa
AlertaStock	gestion.alerta_stock	Directa
ControlMensual	clinico.control_mensual	Directa
ReporteEvolucion	clinico.reporte_evolucion (VISTA, no tabla)	Materializa literalmente la nota original: “vista derivada, no persistida” (ver nota E)
HexagonoCobertura (H3Cell)	geo.hexagono_cobertura	Directa, se preserva explícitamente la ausencia de FK a Paciente (RN-08)
UsuarioSistema	gestion.usuario_sistema	Directa
Rol	gestion.rol	Directa
(sin clase equivalente)	clinico.paciente_cuidador	Tabla nueva: resuelve la relación M:N Paciente-Cuidador, no representable como tabla propia en un diagrama de clases UML (allí es solo una asociación)
(sin clase equivalente)	clinico.recurso_fhir	Tabla nueva (técnica): resuelve la referencia polimórfica de AuditEvent con integridad real (ver nota C)
(sin clase equivalente)	geo.microrred, geo.establecimiento_salud	Tablas nuevas: la organización territorial estaba implícita (“establecimientoAsignado” como atributo de tipo texto en Paciente) pero nunca modelada como entidad propia; CE022 exige explícitamente su normalización
(sin clase equivalente)	gestion.agente_comunitario	Tabla nueva: exigida por el alcance explícito de CE022 (“agentes comunitarios”); no existía como clase de software porque el Entregable N.º 5 solo modeló actores que operan directamente el sistema

Nota A (Paciente — nombres/apellidos). El Diagrama 6 no listaba un atributo de nombre en **Paciente**. Este modelo lo añade porque es un requisito operativo real e ineludible de RF-04 (búsqueda del niño/a en la agenda del día, pantalla P04): sin un campo de nombre, la pantalla P03 (“Agenda del día”) no podría mostrar la lista “Niño A — CRED 6 meses” que el propio prototipo del Entregable N.º 5 diseñó. Se documenta como una extensión necesaria, no como una corrección de un error del entregable anterior.

Nota B (Cuidador — reubicación de parentesco). El Diagrama 6 listaba **parentesco** como atributo directo de **Cuidador**. Este modelo lo reubica como atributo de la tabla de unión **paciente_cuidador**, porque el parentesco es una propiedad de la **relación** entre un cuidador

y un paciente específico, no del cuidador en sí: una misma persona puede ser madre de un niño/a y tía de otro registrado en el mismo sistema. Este es un refinamiento propio de la normalización relacional (evita que el mismo cuidador con dos parentescos distintos requiera dos filas duplicadas de `Cuidador`), no posible de expresar con la misma precisión en un diagrama de clases simple.

Nota C (AuditEvent — integridad referencial real de la referencia polimórfica). La sección 4.5 del Entregable N.º 5 estableció que `AuditEvent` “puede referenciar a cualquiera de los otros cinco recursos, sin excepción” (RN-11). Una traducción ingenua de esta regla a SQL usaría una columna `recurso_id` UUID sin llave foránea (validada solo en la capa de aplicación), lo que **no** cumpliría el criterio de “integridad referencial completa” de la rúbrica de este entregable. Este modelo introduce en su lugar `clinico.recurso_fhir`, una tabla técnica cuya clave primaria comparten `paciente`, `consentimiento`, `dispositivo`, `determinacion` y `provenance` (patrón de diseño conocido como *Class Table Inheritance* o “superclase de identificadores compartidos”): `auditevent.recurso_id` es una FK real hacia `recurso_fhir.id`, por lo que el motor de base de datos garantiza, sin posibilidad de excepción, que todo `AuditEvent` apunta a un recurso que efectivamente existe — sin necesitar cinco columnas de FK nulificables (una trampa común de modelado que viola la Tercera Forma Normal al introducir columnas casi siempre nulas).

Nota D (Provenance — reconciliación de cardinalidad). El Diagrama 6 dibuja, por una decisión de layout, una única relación de `CasoConfirmacion` hacia `Provenance` con cardinalidad 1:1. Sin embargo, el texto de la sección 4.5 del mismo Entregable N.º 5 es más preciso y prevalece sobre el dibujo: “*Provenance / CorreccionAltitud (agente = sistema/ algoritmo), CasoConfirmacion (agente = Personal médico) [...] Siempre apunta a un Observation*”. Esto implica que puede existir **más de un** registro de `Provenance` por `Determinacion` (uno de origen algorítmico, generado siempre; y, si el caso se deriva y se confirma, un segundo de origen humano). Este modelo implementa fielmente el texto (cardinalidad `determinacion 1 : provenance 1..*`, con `determinacion_id` NOT NULL), documentando explícitamente esta reconciliación para que no se lea como una divergencia no intencional.

Nota E (ReporteEvolucion — de clase a vista). El Diagrama 6 ya anotó esta clase como “vista derivada, no persistida como recurso FHIR propio”. Este modelo cumple esa intención literalmente: `clinico.reporte_evolucion` se implementa como una sentencia `CREATE VIEW` (Sección 2.2), no como una tabla con datos propios, evitando la duplicación de `hb_observada/hb_corregida/severidad` que ya existen en `correccion_altitud` y `determinacion`.

Nota de coherencia con ADR-06 (Location / establecimiento_salud). El ADR-06 del Entregable N.º 5 fijó el alcance de recursos FHIR **clínicos** de Fase 0 en exactamente seis (Patient, Consent, Device, Observation, Provenance, AuditEvent) y difirió explícitamente un séptimo (MedicationRequest) a Fase 1. La tabla `geo.establecimiento_salud` introducida en este entregable **no contradice ese alcance**: no es un recurso FHIR clínico y no se cuenta dentro de los “seis” ya comprometidos. Es, en cambio, el equivalente funcional de un recurso FHIR *Location* — una entidad de apoyo administrativo-geográfico transversal en el estándar HL7 FHIR, no considerada parte del núcleo clínico de ningún proyecto—, que **podría** serializarse como `Location` si en el futuro se requiere para interoperabilidad con el HIS-MINSA (por ejemplo, referenciada desde `Patient.managingOrganization`), pero que en esta Parte 1 existe únicamente como tabla de apoyo del modelo relacional. Esta distinción se dedica una nota explícita porque el proyecto exige, como práctica consistente, verificar que ningún entregable nuevo contradiga los diez ADR ya fijados.

1.5 Modelo lógico relacional

La siguiente tabla resume, esquema por esquema, la estructura lógica de cada tabla: su llave primaria, sus llaves foráneas salientes y las restricciones distintivas que la caracterizan (el detalle

exhaustivo columna por columna se desarrolla en el diccionario de datos, sección 1.7). Esta vista intermedia —entre el diagrama conceptual (1.3) y el diccionario completo (1.7)— es la que permite verificar de un vistazo que ninguna tabla queda sin llave primaria y que toda relación declarada en los diagramas E-R tiene su llave foránea correspondiente.

Esquema gestion

Tabla	PK	FK salientes	Restricciones distintivas
rol	id (identity)	—	UNIQUE(nombre_rol); CHECK de 7 valores admitidos
usuario_sistema	id (UUID)	rol_id -> rol; establecimiento_asignacion_id -> establecimiento_salud	UNIQUE(correo); UNIQUE(keycloak_subject_id)
agente_comunitario	id (UUID)	establecimiento_referencia_id -> establecimiento_salud; supervisado_por_usuario_id -> usuario_sistema	
alerta_stock	id (UUID)	establecimiento_id -> establecimiento_salud	CHECK de estado y de valores no negativos

Esquema geo

Tabla	PK	FK salientes	Restricciones distintivas
microrred	id (UUID)	—	CHECK(ubigeo_departamento = '21')
establecimiento_salud	id (UUID)	microrred_id -> microrred	CHECK de categoría (I-1..II-2); índice GIST sobre ubicacion
hexagono_cobertura	id (UUID)	microrred_id -> microrred	UNIQUE(h3_index, periodo_inicio, periodo_fin); sin FK a paciente/determinacion (RN-08)

Esquema config

Tabla	PK	FK salientes	Restricciones distintivas
modelo_inferencia	id (UUID)	—	UNIQUE(version)
configuracion_clinica	id (UUID)	aprobado_por_usuario_id -> gestion.usuario_sistema	UNIQUE(tipo_parametro, version); índice único parcial uq_config_vigente (una sola versión vigente por tipo — materializa RN-02/CA-10)

Esquema clinico

Tabla	PK	FK salientes	Restricciones distintivas
recurso_fhir	id (UUID)	—	CHECK de 5 tipos admitidos (superclase técnica, ver nota C, 1.4)
paciente	id (UUID, = FK a recurso_fhir)	id -> recurso_fhir; establecimiento_asignado -> establecimiento_salud	UNIQUE(codigo_padron_nominal); CHECK de edad (< 6 años)
cuidador	id (UUID)	—	CHECK de idioma (RNF-15)
paciente_cuidador	(paciente_id, cuidador_id) compuesta	paciente_id -> paciente; cuidador_id -> cuidador	ON DELETE CASCADE (tabla de unión pura)
consentimiento	id (UUID, = FK a recurso_fhir)	paciente_id -> paciente; cuidador_id -> cuidador	índice único parcial uq_consentimiento_vigente_por_paciente
dispositivo	id (UUID, = FK a recurso_fhir)	establecimiento_asignado -> establecimiento_salud	UNIQUE(numero_serie)
captura	id (UUID)	paciente_id -> paciente; dispositivo_id -> dispositivo; capturado_por_usuario_id -> gestion.usuario_sistema	CHECK(array_length(espectro_as7341, 1) > 0) índice GIST sobre coordenada_gps
correccion_altitud	id (UUID)	captura_id -> captura (UNIQUE, fuerza 1:1); configuracion_clinica_id -> config.configuracion_clinica	CHECK de rangos de altitud y de Hb
determinacion	id (UUID, = FK a recurso_fhir)	paciente_id -> paciente; correccion_altitud_id -> correccion_altitud (UNIQUE); dispositivo_id -> dispositivo; modelo_inferencia_id -> config.modelo_inferencia	CHECK de severidad (4 valores) y de estado (9 valores, igual al Diagrama 11)
alerta_derivacion	id (UUID)	determinacion_id -> determinacion (UNIQUE)	CHECK de severidad y estado

Tabla	PK	FK salientes	Restricciones distintivas
caso_confirmacion	id (UUID)	alerta_derivacion_id -> alerta_derivacion (UNIQUE); profesional_usuario_id -> gestion.usuario_sistema	CHECK de resultado y modalidad
prescripcion	id (UUID)	caso_confirmacion_id -> caso_confirmacion (UNIQUE); alerta_stock_id -> gestion.alerta_stock	—
provenance	id (UUID, = FK a recurso_fhir)	determinacion_id -> determinacion (NOT NULL, no UNIQUE); agente_usuario_id -> gestion.usuario_sistema	CHECK de origen y tipo de agente
auditevent	id (UUID)	recurso_id -> recurso_fhir; actor_usuario_id -> gestion.usuario_sistema	CHECK de acción (CRUD) y resultado
control_mensual	id (UUID)	paciente_id -> paciente; determinacion_id -> determinacion (UNIQUE, nutable)	UNIQUE(paciente_id, numero_control)

Vista derivada: clinico.reporte_evolucion (sin PK propia; SELECT sobre paciente + determinacion + correccion_altitud).

1.6 Modelo normalizado: prueba de Primera, Segunda y Tercera Forma Normal

Se demuestra el cumplimiento de 1FN, 2FN y 3FN sobre cuatro tablas representativas —elegidas por cubrir los casos más exigentes del esquema (llave compuesta, llave que no es UUID simple, columna de tipo arreglo y columna JSONB)— y se declara que la misma disciplina de diseño se aplicó, sin excepción, al resto de las 24 tablas.

clinico.paciente_cuidador (caso de llave compuesta). - *1FN*: todos los atributos (paciente_id, cuidador_id, parentesco, es_contacto_principal) son atómicos y de un solo valor; no hay grupos repetitivos. Cumple. - *2FN*: la llave primaria es compuesta (paciente_id, cuidador_id). parentesco y es_contacto_principal dependen de **ambas** columnas de la llave a la vez (el parentesco es una propiedad del par específico paciente-cuidador, no de uno solo) — no existe dependencia parcial de solo una parte de la llave. Cumple. - *3FN*: no existen dependencias transitivas (ningún atributo no clave determina a otro atributo no clave). Cumple.

clinico.determinacion (caso de núcleo clínico con múltiples FK). - *1FN*: atributos atómicos (valor_hb_final, severidad, estado, fecha_creacion); ningún atributo agrupa múltiples valores. Cumple. - *2FN*: la llave primaria es simple (id), por lo que no aplica el caso de dependencia parcial (solo es relevante con llaves compuestas); se cumple trivialmente. - *3FN*: se verificó explícitamente que ningún atributo no clave depende de otro atributo

to no clave. Un caso límite discutido y descartado: **severidad** podría *parecer* derivable de **valor_hb_final** mediante los cortes de **config.configuracion_clinica**, lo que sugeriría una dependencia transitiva. Se decidió **mantener severidad como columna propia** (no derivarla en tiempo de consulta) porque los cortes de severidad son ellos mismos versionados en el tiempo (**config.configuracion_clinica**); si se recalculara **severidad** en cada consulta contra la versión *vigente* de los cortes, una determinación histórica cambiaría retroactivamente de clasificación cada vez que se actualice la tabla de cortes, lo cual violaría directamente la trazabilidad exigida por RN-02 y CA-10 (“mantener trazabilidad de qué versión se aplicó a cada determinación histórica”). Esta es una **desnormalización deliberada y documentada**, no un descuido: se prioriza la inmutabilidad histórica exigida por una regla de negocio explícita sobre la pureza normalizadora estricta.

clinico.captura (caso de columna de tipo arreglo). - *1FN (discusión explícita)*: la columna **espectro_as7341** NUMERIC(10,4)[] es, en sentido estricto de la teoría relacional clásica, un caso límite de 1FN, que exige valores atómicos y prohíbe “grupos repetitivos”. La alternativa estrictamente normalizada sería una tabla hija **captura_canal_espectral**(**captura_id**, **indice_canal**, **valor**) con 11 filas por captura. Se evaluaron ambas opciones y se optó deliberadamente por el arreglo de tamaño fijo, por tres razones documentadas: (i) los 11 canales del sensor AS7341 siempre se leen y se escriben como una unidad atómica de dominio — nunca se consulta o actualiza un canal de forma aislada de los otros diez —, por lo que conceptualmente se comportan como un único valor compuesto (análogo a como **coordinada_gps** **geography**(Point) también es, en rigor, un par de coordenadas, pero se trata como un tipo atómico de dominio geoespacial); (ii) PostgreSQL ofrece soporte nativo, indexable y con validación de longitud (**array_length(...)** = 11) para este patrón, sin las siete llaves foráneas adicionales y el JOIN obligatorio que exigiría la alternativa normalizada; y (iii) el propio Entregable N.º 5 (RF-02) define el espectro como una unidad de captura sincronizada de “11 canales”, reforzando que el dominio del problema ya lo trata como un valor compuesto único. Se documenta esta decisión explícitamente como una **excepción justificada y acotada** a la 1FN estricta, no como un patrón a repetir libremente en el resto del esquema (ninguna otra tabla usa arreglos). - *2FN y 3FN*: con llave primaria simple (**id**) y sin dependencias transitivas entre **paciente_id**, **dispositivo_id**, **marca_tiempo**, **calidad_senal**, etc., ambas formas se cumplen sin excepción.

config.configuracion_clinica (caso de columna JSONB). - *1FN (discusión explícita)*: **valor_versionado** JSONB almacena una estructura semi-estructurada (p. ej. los tramos de la fórmula de corrección por altitud), que podría interpretarse como una violación de la atomicidad de 1FN. Se justifica de forma análoga al caso anterior: el **contenido interno** de esa estructura todavía no está definido con exactitud (depende del texto íntegro, aún pendiente, de la NTS 213-MINSA/DGIESP-2024 — RN-02), por lo que fijar columnas rígidas para “tramo 1”, “tramo 2”, etc. sería una normalización prematura sobre datos cuya forma exacta se desconoce. JSONB se usa aquí exactamente para el propósito que la regla de negocio RN-02 exige: permitir que el parámetro sea “parametrizable, versionado y auditable” sin fijar su forma en el esquema. Esta misma justificación aplica a **modelo_inferencia.metricas**, **gestion.rol.permisos**, **clinico.consentimiento.alcance**, **clinico.prescripcion.anotacion_estructurada** y **clinico.auditever**. - *2FN y 3FN*: se cumplen sin excepción (llave simple, sin dependencias transitivas entre **tipo_parametro**, **version**, **fecha_vigencia_desde/hasta** y **aprobado_por_usuario_id**).

Declaración general. Las 20 tablas restantes del esquema no presentan ninguno de los dos casos límite discutidos (llave compuesta con dependencia parcial potencial, o columnas de tipo no escalar) y cumplen 1FN, 2FN y 3FN de forma directa: cada una tiene una llave primaria simple de tipo UUID (o SMALLINT IDENTITY en el caso de **rol**), todos sus atributos no clave dependen funcionalmente de esa llave completa, y no se identificó ninguna dependencia transitiva entre atributos no clave.

1.7 Diccionario de datos completo

Se documentan las 24 tablas y la vista derivada, agrupadas por esquema en el mismo orden de creación del script DDL (Sección 2.2, Anexo A). Columnas: **Nulo** indica si la columna admite NULL (No = NOT NULL); **Clave** indica PK (llave primaria), FK->tabla (llave foránea y su tabla referenciada) o UQ (restricción UNIQUE no relacionada a llave).

Esquema gestion

gestion.rol — Catálogo RBAC (RF-37, RNF-06); la fuente de verdad de autenticación es Keycloak/OIDC (ADR-09).

Columna	Tipo	Nulo	Clave	Descripción
id	SMALLINT (identity)	No	PK	Identificador autogenerado
nombre_rol	VARCHAR(30)	No	UQ	Uno de 7 valores: personal_cred, personal_medico, farmacia, jefatura_microred, analista_diresa, administrador, auditor
descripcion	VARCHAR(200)	No	—	Descripción funcional del rol
permisos	JSONB	No	—	Lista de permisos granulares (ver nota JSONB, sección 1.6)

gestion.usuario_sistema — Usuario interno (personal CRED, médico, farmacia, jefatura, analista, administrador, auditor).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
nombre_completo	VARCHAR(150)	No	—	Nombre del usuario (dato ilustrativo en Fase 0)
correo	VARCHAR(150)	No	UQ	Correo institucional
rol_id	SMALLINT	No	FK->gestion.rol	Rol RBAC asignado
establecimiento_asignado_id	UUID	Sí	FK->geo.establecimiento	Establecimiento de salud
keycloak_subject_id	VARCHAR(100)	Sí	UQ	Enlace al sub del token OIDC (ADR-09)
estado_cuenta	VARCHAR(12)	No	—	activo / inactivo / suspendido
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica de creación del registro

gestion.agente_comunitario — Agente Comunitario de Salud (ACS); entidad nueva de este entregable (ver nota, sección 1.4).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
nombre_completo	VARCHAR(150)	No	—	Nombre del agente comunitario
documento_identidad	VARCHAR(128)	Sí	—	Hash del documento de identidad, nunca en claro (Ley N.° 29733); mecanismo exacto [DATO PENDIENTE DE VALIDAR]
establecimiento_referencia	UUID	No	FK- >geo.establecimiento_salud	Establecimiento al que se refiere
supervisado_por_usuario	UUID	Sí	FK- >gestion.usuario_sistema	Personal CRED que supervisa
telefono_contacto	VARCHAR(20)	Sí	—	Contacto
zona_cobertura_descripcion	VARCHAR(200)	Sí	—	Descripción textual de su zona de cobertura
fecha_capacitacion	DATE	Sí	—	Última capacitación registrada
estado_activo	BOOLEAN	No	—	Activo/inactivo
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

gestion.alerta_stock — Alerta de quiebre de stock de insumos de hierro (RF-25, RF-26).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
establecimiento_id	UUID	No	FK- >geo.establecimiento_salud	Establecimiento al que se refiere
insumo	VARCHAR(100)	No	—	Insumo (por defecto “Sulfato ferroso”)
stock_proyectado	NUMERIC(10,2)	No	—	Consumo proyectado
stock_disponible	NUMERIC(10,2)	No	—	Stock actual
estado	VARCHAR(20)	No	—	abierta / atendida / descartada
fecha_generacion	TIMESTAMPTZ	No	—	Fecha de generación de la alerta
fecha_resolucion	TIMESTAMPTZ	Sí	—	Fecha de cierre, si aplica

Esquema geo

geo.microrred — Unidad organizacional MINSA; caso ilustrativo “Microred de Salud Altiplano-Puno”.

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
nombre	VARCHAR(150)	No	—	Nombre de la microred
red_salud	VARCHAR(150)	No	—	Red de salud a la que pertenece (referencial)
diresa_geresa	VARCHAR(150)	No	—	DIRESA/GERESA responsable (referencial)
ubigeo_departamento	CHAR(2)	No	—	21 = Puno (único valor válido)
ubigeo_provincia	CHAR(4)	Sí	—	[DATO PENDIENTE DE VALIDAR: exacto de la microred real]
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

geo.establecimiento_salud — Establecimiento de salud; equivalente funcional a FHIR *Location* (ver nota ADR-06, sección 1.4).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
microrred_id	UUID	No	FK-> geo.microrred	Microrred a la que pertenece
nombre	VARCHAR(150)	No	—	Nombre del establecimiento
categoria	VARCHAR(10)	No	—	I-1, I-2, I-3, I-4, II-1 o II-2
codigo_renaes	VARCHAR(20)	Sí	—	[DATO PENDIENTE DE VALIDAR]
ubigeo	CHAR(6)	Sí	—	[DATO PENDIENTE DE VALIDAR: distrital exacto]
ubicacion	geography(Point,4326)	No	—	Coordenada PostGIS (punto de tamizaje fijo del establecimiento)
altitud_referencia_msnm	NUMERIC(6,1)	No	—	Altitud oficial, derivada de MDE — nunca de GPS crudo (RN-03)
fuelle_altitud	VARCHAR(20)	No	—	MDE (default) o GPS_provisional

Columna	Tipo	Nulo	Clave	Descripción
es_cabecera_microrred	BOOLEAN	No	—	Marca el establecimiento sede jefatural Auditoría técnica
creado_en	TIMESTAMPTZ	No	—	

geo.hexagono_cobertura — Capa agregada H3 (RF-33, RF-34); materializa RN-08 en el esquema.

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
h3_index	VARCHAR(16)	No	—	Índice H3 (Uber), formato hexadecimal
resolucion_h3	SMALLINT	No	—	Resolución H3 (5 a 9); por defecto 7
microrred_id	UUID	No	FK- >geo.microrred	Microrred de referencia
centroide	geography(Point,4326)	Sí	—	Centroide del hexágono (para renderizado en el mapa)
periodo_inicio	DATE	No	—	Inicio del periodo agregado
periodo_fin	DATE	No	—	Fin del periodo agregado
tamizajes_total	INTEGER	No	—	Conteo agregado (nunca identificadores individuales)
derivaciones_total	INTEGER	No	—	Conteo agregado de derivaciones
prevalencia_estimada	NUMERIC(5,2)	Sí	—	Porcentaje estimado
ultima_actualizacion	TIMESTAMPTZ	No	—	Última corrida del proceso ETL que la actualizó

Columna	Tipo	Nulo	Clave	Descripción
---------	------	------	-------	-------------

Esquema config

`config.modelo_inferencia` — Versión del modelo de inferencia embebido (RF-11, ADR-04).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
version	VARCHAR(20)	No	UQ	Versión semántica del modelo
formato	VARCHAR(10)	No	—	ONNX o TFLite
fecha_entrenamiento	DATE	Sí	—	Fecha de entrenamiento
metricas	JSONB	No	—	Métricas de desempeño (p. ej. MAE); valores exactos [DATO PENDIENTE DE VALIDAR]
activo	BOOLEAN	No	—	Versión vigente distribuida a dispositivos
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

`config.configuracion_clinica` — Parámetros clínicos versionados (RN-02); tabla de corrección por altitud y umbrales de severidad.

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
tipo_parametro	VARCHAR(30)	No	—	<code>tabla_correccion_altitud</code> , <code>umbral_severidad_hb</code> o <code>posologia_hierro</code>
version	VARCHAR(20)	No	UQ (compuesta con <code>tipo_parametro</code>)	Versión del parámetro
valor_versionado	JSONB	No	—	Contenido del parámetro; forma exacta pendiente de NTS 213
fecha_vigencia_desde	TIMESTAMPTZ	No	—	Inicio de vigencia
fecha_vigencia_hasta	TIMESTAMPTZ	Sí	UQ parcial	NULL = versión vigente (a lo sumo una por tipo)
aprobado_por_usuario	UUID	Sí	FK- <code>gestion.usuario_sesiones</code>	Quién aprobó la sesión
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

Esquema clinico

`clinico.recurso_fhir` — Tabla técnica de identificadores compartidos (ver nota C, sección 1.4); no es en sí un recurso FHIR.

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador compartido con la tabla hija correspondiente
tipo_recurso	VARCHAR(20)	No	—	Patient , Consent , Device , Observation o Provenance
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

clinico.paciente — FHIR *Patient*.

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK, FK-> recurso_fhir	Identificador (tipo_recurso ='Patient')
codigo_padron_nominal	VARCHAR(20)	Sí	UQ	Código del Padrón Nominal; nulo si solo hay registro local
nombres	VARCHAR(100)	No	—	Extensión respecto al Diagrama 6 (nota A, 1.4)
apellidos	VARCHAR(100)	No	—	Ídem
fecha_nacimiento	DATE	No	—	Debe ser < 6 años a la fecha de registro
sexo	CHAR(1)	No	—	M o F
establecimiento_asignado_id	UUID	No	FK-> geo.establecimiento_asignado	Establecimiento de atención
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

clinico.cuidador — Madre, padre o tutor.

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
nombre_completo	VARCHAR(150)	No	—	Nombre del cuidador/a
telefono_contacto	VARCHAR(20)	Sí	—	Contacto
idioma_preferido	VARCHAR(20)	No	—	espanol (default), quechua o aymara (RNF-15)
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

clinico.paciente_cuidador — Relación M:N Paciente-Cuidador (tabla nueva, nota 1.4).

Columna	Tipo	Nulo	Clave	Descripción
paciente_id	UUID	No	PK (compuesta), FK-> paciente	—

Columna	Tipo	Nulo	Clave	Descripción
cuidador_id	UUID	No	PK (compuesta), FK->cuidador	—
parentesco	VARCHAR(20)	No	—	madre, padre, abuela, abuelo, tía, tío, hermano mayor, tutor legal, otro
es_contacto_principal	BOOLEAN	No	—	Marca el contacto preferente para notificaciones

`clinico.consentimiento` — FHIR *Consent* (RF-12, RN-04).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK, FK- >recurso_fhir	Identificador (tipo_recurso='Consent')
paciente_id	UUID	No	FK->paciente	Paciente al que aplica
cuidador_id	UUID	No	FK->cuidador	Firmante
estado	VARCHAR(10)	No	—	vigente o revocado
fecha_otorgamiento	TIMESTAMPTZ	No	—	Fecha de firma
fecha_revocacion	TIMESTAMPTZ	Sí	—	Obligatoria si estado='revocado' (CHECK)
alcance	JSONB	No	—	Qué datos se capturan, finalidad, base legal
firma_digital_hash	VARCHAR(128)	No	—	Hash de la firma digital
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

`clinico.dispositivo` — FHIR *Device* (nodo HemoScan Andino).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK, FK- >recurso_fhir	Identificador (tipo_recurso='Device')
numero_serie	VARCHAR(40)	No	UQ	Número de serie físico del nodo
version_firmware	VARCHAR(20)	No	—	Versión de firmware (RF-11)
fecha_ultima_calibracion	DATE	Sí	—	Última calibración
estado	VARCHAR(20)	No	—	operativo, en_calibracion o fuera_de_servicio
establecimiento_asignado	UUID	Sí	FK- >geo.establecimiento_asignado	Establecimiento actual
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

`clinico.captura` — Captura sincronizada multisensor (RF-02, RF-03).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
paciente_id	UUID	No	FK-> paciente	Paciente capturado
dispositivo_id	UUID	No	FK-> dispositivo	Nodo utilizado
capturado_por_usuario_id	UUID	Sí	FK-> gestion.usuario	Personal CRED que capturó la captura
marca_tiempo	TIMESTAMPZ	No	—	Momento exacto de la captura
imagen_conjuntiva_marcha	VARBINARY(300)	No	—	Referencia URI/blob externo (no BYTEA, ver 2.2)
espectro_as7341	NUMERIC(10,4)[[]]	No	—	Arreglo de exactamente 11 canales (CHECK)
senal_ppg_ref	VARBINARY(300)	No	—	Referencia URI/blob externo
coordenada_gps	geography(Point,4326)	No	—	“Punto de tamizaje” PostGIS; nunca su altitud implícita (RN-03)
calidad_senal	NUMERIC(4,3)	No	—	Entre 0 y 1 (RF-04)
intentos_recaptura	SMALLINT	No	—	0 a 3
creado_en	TIMESTAMPZ	No	—	Auditoría técnica

`clinico.correccion_altitud` — Corrección de hemoglobina por altitud (RF-07 a RF-09, ADR-05).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
captura_id	UUID	No	UQ, FK-> captura	Fuerza cardinalidad 1:1 con captura
configuracion_clinica_id	UUID	No	FK-> config.configuracion_clinica	Versión de función aplicada
altitud_mde	NUMERIC(6,1)	No	—	Altitud derivada del MDE en el punto de captura
fuentes_mde	VARBINARY(30)	No	—	Identificación de la fuente del MDE
hb_observada	NUMERIC(4,1)	No	—	Hemoglobina “cruda” estimada (antes de corregir)
hb_corregida	NUMERIC(4,1)	No	—	Hemoglobina “ajustada” (después de corregir)

Columna	Tipo	Nulo	Clave	Descripción
formula_version	VARCHAR(20)	No	—	Versión de la fórmula aplicada (redundante legible con FK, por auditoría directa)
timestamp	TIMESTAMPTZ	No	—	Momento del cálculo

`clinico.determinacion` — FHIR *Observation* (núcleo clínico; RF-10, RF-28).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK, FK->recurso_fhir	Identificador (tipo_recurso='Observation')
paciente_id	UUID	No	FK->paciente	Paciente evaluado
correccion_altitud_id	UUID	No	UQ, FK->correccion_altitud	Fuerza 1:1
dispositivo_id	UUID	No	FK->dispositivo	Nodo de origen
modelo_inferencia_id	UUID	No	FK->config.modelo_inferencia	Versión de modelo inferencia (RF-11)
valor_hb_final	NUMERIC(4,1)	No	—	Valor final reportado (= hb_corregida)
severidad	VARCHAR(12)	No	—	sin_anemia, leve, moderada, severa
estado	VARCHAR(30)	No	—	9 valores, réplica literal del Diagrama 11 (Entregable N.º 5)
fecha_creacion	TIMESTAMPTZ	No	—	Momento de creación

`clinico.alerta_derivacion` — Alerta priorizada por severidad (RF-14, RN-01).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
determinacion_id	UUID	No	UQ, FK->determinacion	Fuerza 1:1
severidad	VARCHAR(12)	No	—	leve, moderada, severa
estado	VARCHAR(15)	No	—	pendiente, atendida, vencida
fecha_generacion	TIMESTAMPTZ	No	—	—
fecha_atencion	TIMESTAMPTZ	Sí	—	Cuándo se atendió

`clinico.caso_confirmacion` — Confirmación/descarte clínico humano obligatorio (RF-21, RF-22, RN-06).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
alerta_derivacion_id	UUID	No	UQ, FK->alerta_derivacion	Fuerza 1:1

Columna	Tipo	Nulo	Clave	Descripción
profesional_usuario_id	UUID	No	FK- >gestion.usuario_personal_medico	Debe tener rol personal_medico (RN-06; validación de aplicación en Parte 1, trigger en Parte 2)
resultado	VARCHAR(12)	No	—	confirmado o descartado
modalidad	VARCHAR(15)	No	—	presencial o telemedicina (ADR-07, no activo en Fase 0)
fecha_decision	TIMESTAMPTZ	No	—	—

clinico.prescripcion — Prescripción de hierro como anotación estructurada de Observation (ADR-06).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
caso_confirmacion_id	UUID	No	UQ, FK- >caso_confirmacion	Fuerza 1:1
alerta_stock_id	UUID	Sí	FK- >gestion.alerta_stock	Si la prescripción coincide con una alerta de stock
tipo_tratamiento	VARCHAR(50)	No	—	Por defecto sulfato_ferroso
anotacion_estructurada	JSONB	No	—	Dosis/vía; posología exacta [DATO PENDIENTE DE VALIDAR]
fecha_inicio	DATE	No	—	—
duracion_dias_estimada	SMALLINT	Sí	—	—
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

clinico.provenance — FHIR *Provenance* (ver nota D, sección 1.4).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK, FK- >recurso_fhir	Identificador (tipo_recurso='Provenance')
determinacion_id	UUID	No	FK- >determinacion	Objetivo (<i>target</i>); nunca nulo
origen_evento	VARCHAR(20)	No	—	correccion_altitud o confirmacion_clinica
agente_tipo	VARCHAR(20)	No	—	sistema_algoritmo, profesional_salud o dispositivo
agente_usuario_id	UUID	Sí	FK- >gestion.usuario_personal_medico	Obligatorio si agente_tipo='profesional_salud' (CHECK)

Columna	Tipo	Nulo	Clave	Descripción
actividad	VARCHAR(100)	No	—	Descripción de la actividad registrada
timestamp	TIMESTAMPTZ	No	—	—

`clinico.auditevent` — FHIR *AuditEvent* (RF-31, RF-38, RN-11).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
recurso_id	UUID	No	FK-> <code>recurso_fhir</code>	Recurso afectado (cualquiera de los 5 tipos admitidos)
actor_usuario_id	UUID	No	FK-> <code>gestion.usuario_sistema</code>	Quién ejecutó la acción
accion	VARCHAR(10)	No	—	<code>create</code> , <code>read</code> , <code>update</code> , <code>delete</code>
resultado	VARCHAR(10)	No	—	<code>exito</code> o <code>fallo</code>
timestamp	TIMESTAMPTZ	No	—	—
detalle	JSONB	Sí	—	Detalle adicional opcional

`clinico.control_mensual` — Seguimiento mensual del tratamiento (CU-06, RF-16, RF-17, RF-20).

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	No	PK	Identificador
paciente_id	UUID	No	FK-> <code>paciente</code>	—
numero_control	SMALLINT	No	UQ (compuesta con <code>paciente_id</code>)	1 a 6
fecha_programada	DATE	No	—	—
fecha_realizada	DATE	Sí	—	—
estado_asistencia	VARCHAR(15)	No	—	<code>programado</code> , <code>asistio</code> , <code>inasistencia</code> , <code>reprogramado</code>
determinacion_id	UUID	Sí	UQ, FK-> <code>determinacion</code>	Determinación generada por ese control, si se realizó
creado_en	TIMESTAMPTZ	No	—	Auditoría técnica

Vista `clinico.reporte_evolucion` — Historial longitudinal derivado (RF-20); corresponde a la clase `ReporteEvolucion` del Diagrama 6 (nota E, sección 1.4). Columnas expuestas: `paciente_id`, `determinacion_id`, `fecha_creacion`, `hb_observada`, `hb_corregida`, `altitud_mde`, `severidad`, `estado` — todas derivadas por JOIN de `paciente` + `determinacion` + `correccion_altitud`, sin almacenamiento propio.

1.8 Síntesis de integridad referencial y de rendimiento

La siguiente tabla resume, como cierre de la Sección 1 y puente hacia la evidencia de ejecución real de la Sección 2, el inventario cuantitativo de mecanismos de integridad y rendimiento del modelo. Estos números **no son estimados**: se obtuvieron consultando el catálogo del sistema (`pg_constraint`, `pg_indexes`) de la instancia real donde se ejecutó el script DDL (Sección 2.3), y la salida cruda de esa consulta al catálogo queda archivada, sin editar, en `Anexos-Ejecutables-Entregable-06/log_05_conteo_restricciones.txt` (ver “Evidencia externa y reproducibilidad” al inicio de los Anexos).

Mecanismo	Cantidad	Propósito predominante
Llaves primarias (PRIMARY KEY)	24	Una por tabla, sin excepción
Llaves foráneas (FOREIGN KEY)	39	Integridad referencial entre las 4 esquemas
Restricciones CHECK	40	Integridad de dominio (rangos, enumeraciones, coherencia entre columnas)
Restricciones/índices UNIQUE	14	Incluye 2 índices únicos parciales (RN-04 y RN-02/CA-10)
Índices totales	60	Incluye 3 índices GIST sobre columnas <code>geography</code> (PostGIS)

Dos de los catorce mecanismos UNIQUE merecen mención aparte por materializar una regla de negocio directamente en el motor de base de datos, no solo en la capa de aplicación: `uq_consentimiento_vigente_por_paciente` (índice único parcial sobre `consentimiento(paciente_id)` WHERE `estado='vigente'`, que hace **físicamente imposible** que un paciente tenga dos consentimientos vigentes simultáneos, materializando RN-04) y `uq_config_vigente` (índice único parcial análogo sobre `configuracion_clinica(tipo_parametro)` WHERE `fecha_vigencia_hasta IS NULL`, que materializa el requisito de CA-10 de que exista una única versión vigente por tipo de parámetro clínico en todo momento). La Sección 2.6 demuestra, con salida real del motor, que ambos mecanismos efectivamente rechazan los intentos de violación.

2. Implementación de Base de Datos

2.1 Entorno y metodología de validación

A diferencia de los cinco entregables anteriores del proyecto —todos ellos, por diseño y por etapa, ejercicios de gabinete—, esta sección documenta una **ejecución real, fechada y verificable de forma independiente**. El entorno utilizado fue:

Componente	Valor real verificado
Motor de base de datos	PostgreSQL 16.4 (Debian 16.4-1.pgdg110+2), imagen oficial <code>postgis/postgis:16-3.4</code>
Digest inmutable de la imagen	<code>sha256:44126d872ac91993766c341e369c539e8196614321765d36a</code>
Extensión geoespacial	PostGIS 3.4.3 , GEOS 3.9.0, PROJ 7.2.1
Orquestación	Contenedor Docker local (<code>docker compose up -d</code> con el <code>docker-compose.yml</code> archivado en <code>Anexos-Ejecutables-Entregable-06/</code>), efímero, creado y destruido (<code>docker compose down</code>) únicamente para esta validación

Componente	Valor real verificado
Fecha/hora de la corrida (UTC)	2026-07-07T10:00:18Z
Base de datos	hemoscan_andino
Método de ejecución	psql no interactivo (<code>docker exec -u postgres ... -f <script>.sql</code>), con <code>ON_ERROR_STOP=1</code> para los scripts de carga (cualquier error detiene la ejecución) y sin dicha bandera para el script de pruebas de integridad (Sección 2.6), donde los errores son el resultado esperado

Se reitera, con la misma honestidad ya declarada en la nota metodológica inicial, que este contenedor **no es el servidor de producción de la posta** (responsabilidad del paquete EDT 2.2, Integrante 1): es un entorno de validación desechable, equivalente en propósito a un entorno de *staging* de un solo uso, cuyo único fin es demostrar que el script DDL/DML es sintácticamente correcto, semánticamente consistente y efectivamente ejecutable en el motor ya fijado por el ADR-10 del Entregable N.º 5 — no simular esa demostración por escrito. A diferencia de una simulación redactada, esta corrida generó transcripciones crudas que **no viven únicamente dentro de este Markdown**: los cuatro scripts ejecutados y cada transcripción cruda quedan archivados como archivos independientes, con su checksum SHA-256, en la carpeta hermana **Anexos-Ejecutables-Entregable-06/** (detalle completo, procedimiento de reproducción y las dos cifras que la ejecución real permitió corregir frente a lo narrado, en su **README.md**).

El flujo de validación ejecutado fue: (1) creación de las 4 extensiones/esquemas y las 24 tablas + 1 vista (script DDL, Sección 2.2); (2) carga de un conjunto de datos ilustrativo (script DML, Sección 2.4); (3) ejecución de 10 consultas de verificación, incluidas dos consultas geoespaciales PostGIS (Sección 2.5); (4) ejecución de 7 pruebas deliberadas de violación de restricciones más 1 control positivo (8 pruebas en total), para demostrar que la integridad referencial y de dominio se aplica realmente y no solo en teoría, y que las restricciones no rechazan indiscriminadamente toda operación (Sección 2.6).

2.2 Script DDL: estructura y fragmentos representativos

El script DDL completo (**94 sentencias** ejecutadas sin errores: 2 `CREATE EXTENSION`, 4 `CREATE SCHEMA`, 24 `CREATE TABLE`, 22 `CREATE INDEX` explícitos, 1 `CREATE VIEW` y 41 `COMMENT ON`; los 60 índices totales reportados en la Sección 1.8 incluyen además los índices implícitos que PostgreSQL crea automáticamente para cada `PRIMARY KEY` y restricción `UNIQUE`) vive, íntegro y verificable por checksum SHA-256, en el archivo externo **Anexos-Ejecutables-Entregable-06/01_ddl_hemoscan.** el **Anexo A** reproduce su estructura y un extracto representativo, no el archivo completo, para no duplicar en este Markdown un contenido que ya tiene una fuente única y verificable. Se presentan aquí los fragmentos más representativos de las decisiones de diseño ya justificadas en la Sección 1, en el mismo orden de ejecución.

Extensiones y esquemas:

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

```
CREATE SCHEMA IF NOT EXISTS gestion;
CREATE SCHEMA IF NOT EXISTS geo;
CREATE SCHEMA IF NOT EXISTS config;
CREATE SCHEMA IF NOT EXISTS clinico;
```

Patrón de superclase técnica para la referencia polimórfica de FHIR (nota C, sec-

ción 1.4):

```
CREATE TABLE clinico.recurso_fhir (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tipo_recurso VARCHAR(20) NOT NULL,
  creado_en   TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT chk_recurso_tipo CHECK (tipo_recurso IN ('Patient', 'Consent', 'Device', 'Observed')));
```

```
CREATE TABLE clinico.paciente (
  id          UUID PRIMARY KEY REFERENCES clinico.recurso_fhir(id) ON DELETE CASCADE,
  codigo_padron_nominal VARCHAR(20) UNIQUE,
  nombres     VARCHAR(100) NOT NULL,
  apellidos   VARCHAR(100) NOT NULL,
  fecha_nacimiento DATE NOT NULL,
  sexo        CHAR(1) NOT NULL,
  establecimiento_asignado_id UUID NOT NULL REFERENCES geo.establecimiento_salud(id) ON DELETE CASCADE,
  creado_en   TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT chk_paciente_sexo CHECK (sexo IN ('M', 'F')),
  CONSTRAINT chk_paciente_fecha_nac CHECK (fecha_nacimiento <= CURRENT_DATE AND fecha_nacimiento >= '2000-01-01'));
```

Columna de arreglo con validación de longitud fija (espectro AS7341 de 11 canales) y tipo geoespacial PostGIS (captura, la tabla con mayor densidad de restricciones del esquema):

```
CREATE TABLE clinico.captura (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  paciente_id UUID NOT NULL REFERENCES clinico.paciente(id) ON DELETE RESTRICT,
  dispositivo_id UUID NOT NULL REFERENCES clinico.dispositivo(id) ON DELETE RESTRICT,
  capturado_por_usuario_id UUID REFERENCES gestion.usuario_sistema(id) ON DELETE SET NULL,
  marca_tiempo TIMESTAMPTZ NOT NULL DEFAULT now(),
  imagen_conjuntiva_ref VARCHAR(300) NOT NULL,
  espectro_as7341 NUMERIC(10,4)[] NOT NULL,
  senal_ppg_ref VARCHAR(300) NOT NULL,
  coordenada_gps geography(Point, 4326) NOT NULL,
  calidad_senal NUMERIC(4,3) NOT NULL,
  intentos_recaptura SMALLINT NOT NULL DEFAULT 0,
  creado_en   TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT chk_captura_espectro_11ch CHECK (array_length(espectro_as7341, 1) = 11),
  CONSTRAINT chk_captura_calidad CHECK (calidad_senal BETWEEN 0 AND 1),
  CONSTRAINT chk_captura_intentos CHECK (intentos_recaptura BETWEEN 0 AND 3))
);
```

```
CREATE INDEX ix_captura_coordenada_gps ON clinico.captura USING GIST(coordenada_gps);
```

Índices únicos parciales que materializan RN-04 y RN-02/CA-10 directamente en

el motor:

```
CREATE UNIQUE INDEX uq_consentimiento_vigente_por_paciente
  ON clinico.consentimiento(paciente_id) WHERE estado = 'vigente';

CREATE UNIQUE INDEX uq_config_vigente
  ON config.configuracion_clinica(tipo_parametro) WHERE fecha_vigencia_hasta IS NULL;
```

Vista derivada (materializa la clase ReporteEvolucion, nota E, sección 1.4):

```
CREATE VIEW clinico.reporte_evolucion AS
SELECT
  p.id AS paciente_id, d.id AS determinacion_id, d.fecha_creacion,
  ca.hb_observada, ca.hb_corregida, ca.altitud_mde, d.severidad, d.estado
FROM clinico.paciente p
JOIN clinico.determinacion d      ON d.paciente_id = p.id
JOIN clinico.correccion_altitud ca ON ca.id = d.correccion_altitud_id
ORDER BY p.id, d.fecha_creacion;
```

2.3 Evidencia real de creación de tablas, relaciones y restricciones

La siguiente salida resume la **transcripción real** de la sesión `psql` que ejecutó el script DDL completo contra la instancia descrita en 2.1 (se omiten las líneas repetitivas `CREATE TABLE / CREATE INDEX / COMMENT` intermedias, ya que cada una se lista en el inventario posterior; no se omite ni edita ningún mensaje de error, porque no se produjo ninguno). La transcripción cruda, completa y sin editar de las 96 líneas de salida real —incluida la única `NOTICE` emitida— se archiva sin resumir en `Anexos-Ejecutables-Entregable-06/log_01_ddl_ejecucion.txt`:

```
$ docker exec -u postgres hemoscan_andino_validacion psql -U hemoscan_admin -d hemoscan_andino
CREATE EXTENSION
psql:/tmp/01_ddl_hemoscan.sql:10: NOTICE:  extension "postgis" already exists, skipping
CREATE EXTENSION
CREATE SCHEMA    (x4: gestion, geo, config, clinico)
...
[94 sentencias ejecutadas -- 0 errores: 2 CREATE EXTENSION, 4 CREATE SCHEMA,
 24 CREATE TABLE, 22 CREATE INDEX, 1 CREATE VIEW, 41 COMMENT ON]
```

Inventario de objetos verificado mediante `\dt` y `\dv` sobre el catálogo real (no una

lista redactada a mano):

List of relations (schema gestion)

Schema	Name	Type
gestion	agente_comunitario	table
gestion	alerta_stock	table
gestion	rol	table
gestion	usuario_sistema	table

(4 rows)

List of relations (schema geo)

Schema	Name	Type
geo	establecimiento_salud	table
geo	hexagono_cobertura	table
geo	microrred	table

(3 rows)

List of relations (schema config)

Schema	Name	Type
config	configuracion_clinica	table
config	modelo_inferencia	table

(2 rows)

List of relations (schema clinico)

Schema	Name	Type
clinico	alerta_derivacion	table
clinico	auditevent	table
clinico	captura	table
clinico	caso_confirmacion	table
clinico	consentimiento	table
clinico	control_mensual	table
clinico	correccion_altitud	table
clinico	cuidador	table
clinico	determinacion	table
clinico	dispositivo	table
clinico	paciente	table
clinico	paciente_cuidador	table
clinico	prescripcion	table
clinico	provenance	table
clinico	recurso_fhir	table

(15 rows)

List of relations (vistas)

Schema	Name	Type
clinico	reporte_evolucion	view

(1 row)

Total verificado por conteo de catálogo: $4 + 3 + 2 + 15 = 24$ tablas + **1 vista**, coincidente

exactamente con el inventario declarado en la Sección 1.2. La salida completa y sin editar de \dt/\dv para los cuatro esquemas se archiva en `Anexos-Ejecutables-Entregable-06/log_06_catalogo.txt`.

Detalle de restricciones de una tabla representativa (clinico.determinacion), obtenido con \d clinico.determinacion directamente del catálogo del motor (nombres de restricción reales, sin abreviar, tal como los generó automáticamente PostgreSQL):

Foreign-key constraints:

```
"determinacion_correccion_altitud_id_fkey" FOREIGN KEY (correccion_altitud_id)
REFERENCES clinico.correccion_altitud(id) ON DELETE RESTRICT
"determinacion_dispositivo_id_fkey" FOREIGN KEY (dispositivo_id)
REFERENCES clinico.dispositivo(id) ON DELETE RESTRICT
"determinacion_id_fkey" FOREIGN KEY (id)
REFERENCES clinico.recurso_fhir(id) ON DELETE RESTRICT
"determinacion_modelo_inferencia_id_fkey" FOREIGN KEY (modelo_inferencia_id)
REFERENCES config.modelo_inferencia(id) ON DELETE RESTRICT
"determinacion_paciente_id_fkey" FOREIGN KEY (paciente_id)
REFERENCES clinico.paciente(id) ON DELETE RESTRICT
```

Referenced by:

```
TABLE "clinico.alerta_derivacion" CONSTRAINT "alerta_derivacion_determinacion_id_fkey" FO
TABLE "clinico.control_mensual" CONSTRAINT "control_mensual_determinacion_id_fkey" FOR
TABLE "clinico.provenance" CONSTRAINT "provenance_determinacion_id_fkey" FOREIGN K
```

Conteo de restricciones consultado directamente sobre pg_constraint (no estimado):

```
SELECT contype AS tipo_restriccion, count(*) FROM pg_constraint c
JOIN pg_namespace n ON n.oid = c.connamespace
WHERE n.nspname IN ('gestion','geo','config','clinico')
GROUP BY contype ORDER BY contype;
```

tipo_restriccion	total
c (CHECK)	40
f (FOREIGN KEY)	39
p (PRIMARY KEY)	24
u (UNIQUE)	14

(4 rows)

```
total_indices
-----
60
```

Salida cruda idéntica, sin editar, archivada en `Anexos-Ejecutables-Entregable-06/log_05_conteo_restricc`

2.4 Script DML: carga de datos ilustrativos

El script DML completo vive, íntegro y verificable por checksum SHA-256, en el archivo externo `Anexos-Ejecutables-Entregable-06/02_dml_hemoscan.sql`; el **Anexo B** reproduce su estructura y un extracto representativo. Carga un conjunto de datos **enteramente ficticio** (ver advertencia en la nota metodológica inicial) que reutiliza, deliberadamente, las mismas etiquetas “Niño A” a “Niño H” ya usadas en los wireframes del Entregable N.º 5 (pantallas P03, P09, P12, P13) y, para el caso del “Niño A”, **los mismos valores exactos** ya mostrados en el wireframe de la pantalla P09/P13 (altitud 3,812 m; versión de corrección v1.2; severidad LEVE resultante), como demostración de que el modelo de datos puede reproducir con exactitud el

escenario ya narrado en el entregable anterior:

```
-- geo.establishment_salud: 1 Centro de Salud I-4 cabecera + 6 Puestos de
-- Salud I-1/I-2, con los mismos nombres ilustrativos del Entregable N.1
INSERT INTO geo.establishment_salud
    (id, microrred_id, nombre, categoria, ubigeo, ubicacion, altitud_referencia_msnm, es_cabe
('20000000-0000-0000-0000-0000000000001', '10000000-0000-0000-0000-0000000000001',
    'Centro de Salud I-4 "Altiplano" (cabecera de microrred)', 'I-4', '210101',
    ST_SetSRID(ST_MakePoint(-70.1181, -15.5000), 4326)::geography, 3824.0, TRUE),
-- ... 6 filas mas (Comunidad A a F) -- ver Anexo B

-- clinico.correccion_altitud: fila del "Niño A", idéntica al wireframe P09/P13
-- del Entregable N.5 (altitud 3,812 m; version v1.2; LEVE)
INSERT INTO clinico.correccion_altitud
    (id, captura_id, configuracion_clinica_id, altitud_mde, hb_observada, hb_corregida, formu
('55000000-0000-0000-0000-0000000000001', '54000000-0000-0000-0000-0000000000001',
    '41000000-0000-0000-0000-0000000000002', 3812.0, 12.5, 10.8, 'v1.2', '2026-07-07T09:14:30-05:
```

El conjunto de datos resultante cubre deliberadamente distintos estados del ciclo de vida de una Determinación (Diagrama 11, Entregable N.º 5): dos casos `pendiente_derivacion` sin decisión médica todavía (Niño A, Niño D — igual que el mockup de la pantalla P13, que muestra la decisión médica *sin marcar*), un caso `en_seguimiento_mensual` con inasistencia registrada (Niño E, igual que la pantalla P12: “*Niño E — no asistió al control de junio*”), un caso `tratamiento_iniciado` con severidad severa, dos casos `sin_anemia` (preventivos) y dos pacientes todavía sin ninguna determinación (solo registrados y con consentimiento vigente) — cubriendo así, con un conjunto pequeño de solo 8 pacientes ilustrativos, prácticamente todo el espacio de estados relevante para las consultas de verificación de la sección 2.5.

Evidencia real de ejecución (transcripción literal, BEGIN...COMMIT atómico de todo el script — demostración básica de uso de transacciones de cliente, no de programación avanzada de base de datos, que corresponde a la Sección 3), idéntica y sin

editar a Anexos-Ejecutables-Entregable-06/log_02_dml_ejecucion.txt:

```
$ docker exec -u postgres hemoscan_andino_validacion psql -U hemoscan_admin -d hemoscan_andino
BEGIN
INSERT 0 7      -- gestion.rol
INSERT 0 1      -- geo.microrred
INSERT 0 7      -- geo.establecimiento_salud
INSERT 0 9      -- gestion.usuario_sistema
INSERT 0 2      -- gestion.agente_comunitario
INSERT 0 1      -- gestion.alerta_stock
INSERT 0 2      -- config.modelo_inferencia
INSERT 0 3      -- config.configuracion_clinica
INSERT 0 8      -- clinico.recurso_fhir (Patient x8)
INSERT 0 8      -- clinico.paciente
INSERT 0 6      -- clinico.cuidador
INSERT 0 8      -- clinico.paciente_cuidador
INSERT 0 9      -- clinico.recurso_fhir (Consent x9)
INSERT 0 9      -- clinico.consentimiento
INSERT 0 3      -- clinico.recurso_fhir (Device x3)
INSERT 0 3      -- clinico.dispositivo
INSERT 0 6      -- clinico.captura
INSERT 0 6      -- clinico.correccion_altitud
INSERT 0 6      -- clinico.recurso_fhir (Observation x6)
INSERT 0 6      -- clinico.determinacion
INSERT 0 4      -- clinico.alerta_derivacion
INSERT 0 2      -- clinico.caso_confirmacion
INSERT 0 2      -- clinico.prescripcion
INSERT 0 8      -- clinico.recurso_fhir (Provenance x8)
INSERT 0 8      -- clinico.provenance
INSERT 0 6      -- clinico.auditevent
INSERT 0 2      -- clinico.control_mensual
INSERT 0 3      -- geo.hexagono_cobertura
COMMIT
```

2.5 Evidencia real de consultas ejecutadas

Se documentan a continuación, con salida literal real, las consultas más representativas de la variedad exigida por la rúbrica (unión múltiple, agregación, geoespacial y trazabilidad de auditoría) de entre las 10 que componen el **Anexo C**; la salida cruda, completa y sin editar de las 10 consultas —incluidas la de conteo total y el historial de versiones, no citadas aquí en el cuerpo por brevedad— queda archivada en Anexos-Ejecutables-Entregable-06/log_03_consultas_ejecucion.txt

Consulta 1 — Vista derivada reporte_evolucion (join de 3 tablas, demuestra que la

vista de la Sección 2.2 funciona):

paciente_id (...0001)	determinacion_id (...0001)	hb_observada	hb_corregida	altitud_m
...0001 (Nino A)	...0001	12.5	10.8	3800
...0002 (Nino B)	...0002	13.6	11.9	3800
...0004 (Nino D)	...0004	12.3	10.6	3800
...0005 (Nino E)	...0005	10.9	9.2	3800
...0006 (Nino F)	...0006	8.4	6.8	3800
...0007 (Nino G)	...0007	13.9	12.2	3800

(6 rows)

Consulta 4 — Agregación de tasa de derivación por establecimiento (GROUP BY + HAVING + FILTER):

```
SELECT es.nombre, count(d.id) AS total_tamizajes,
       count(*) FILTER (WHERE d.severidad <> 'sin_anemia') AS total_con_anemia,
       round(100.0 * count(*) FILTER (WHERE d.severidad <> 'sin_anemia') / NULLIF(count(d.id), 0)) AS tasa_derivacion_pct
FROM geo.establecimiento_salud es
JOIN clinico.paciente p ON p.establecimiento_asignado_id = es.id
JOIN clinico.determinacion d ON d.paciente_id = p.id
GROUP BY es.nombre HAVING count(d.id) > 0 ORDER BY tasa_derivacion_pct DESC;
```

establecimiento	total_tamizajes	total_con_anemia
Puesto de Salud I-2 "Comunidad B" (ilustrativo)	1	
Puesto de Salud I-1 "Comunidad D" (ilustrativo)	1	
Puesto de Salud I-1 "Comunidad C" (ilustrativo)	1	
Centro de Salud I-4 "Altiplano" (cabecera de microred)	2	
Puesto de Salud I-1 "Comunidad E" (ilustrativo)	1	

(5 rows)

Consulta 5 — Consulta geoespacial PostGIS real (ST_Distance sobre tipo geography, no simulada):

```
SELECT e2.nombre, round((ST_Distance(e1.ubicacion, e2.ubicacion) / 1000)::numeric, 2) AS distancia_km_a_cabecera
FROM geo.establecimiento_salud e1 CROSS JOIN geo.establecimiento_salud e2
WHERE e1.es_cabecera_microrred = TRUE AND e2.id <> e1.id
ORDER BY distancia_km_a_cabecera;
```

establecimiento	distancia_km_a_cabecera
Puesto de Salud I-2 "Comunidad B" (ilustrativo)	2.94
Puesto de Salud I-2 "Comunidad A" (ilustrativo)	3.23
Puesto de Salud I-1 "Comunidad C" (ilustrativo)	9.16
Puesto de Salud I-1 "Comunidad F" (ilustrativo)	9.75
Puesto de Salud I-1 "Comunidad D" (ilustrativo)	10.39
Puesto de Salud I-1 "Comunidad E" (ilustrativo)	11.07

(6 rows)

Esta consulta demuestra el uso genuino de PostGIS (ADR-10): `ST_Distance` sobre columnas `geography(Point,4326)` calcula distancias geodésicas reales en metros (convertidas aquí a kilómetros), no distancias euclidianas planas, lo cual es relevante para el caso de uso real RF-27 (sugerir un establecimiento cercano con disponibilidad de stock).

Consulta 6 — Vecino más cercano (ORDER BY <->, operador de distancia indexable)

de PostGIS) a un punto de tamizaje real (captura.coordenada_gps):

nombre	distancia_km
Puesto de Salud I-1 "Comunidad C" (ilustrativo)	0.00
Centro de Salud I-4 "Altiplano" (cabecera de microred)	9.16
Puesto de Salud I-1 "Comunidad E" (ilustrativo)	9.33

(3 rows)

Consulta 7 — Traza de auditoría completa de una determinación (unión AuditEvent + Provenance, RF-31/RF-38/RN-11):

tipo	evento	actor	timestamp
AuditEvent	create	Personal CRED - Altiplano (ilustrativo)	2026-07-07
Provenance	correccion_altitud	sistema_algoritmo	2026-07-07
AuditEvent	read	Personal Medico - Altiplano (ilustrativo)	2026-07-07

(3 rows)

Consulta 9 — Verificación directa de RN-04 (ningún paciente con más de un Consent vigente simultáneo) y del ciclo revocar/re-otorgar (hallazgo H2, Entregable N.º 5):

paciente_id	consentimientos_vigentes
(0 rows)	

-- CERO violaciones, como se espera

paciente_id (Nino C)	estado	fecha_otorgamiento	fecha_revocacion
...0003	revocado	2026-05-10 15:00:00+00	2026-05-20 21:30:00+00
...0003	vigente	2026-06-02 16:00:00+00	

(2 rows)

2.6 Evidencia real de pruebas de integridad referencial y de dominio

Para verificar que la integridad “de papel” del diseño (Sección 1) efectivamente se aplica en el motor, se ejecutaron 7 intentos deliberados de insertar/modificar datos inválidos, más 1 control positivo (8 pruebas en total). El **Anexo D** reproduce el script completo; a continuación, la salida real (literal, simplificada a una línea por prueba) del motor para cada uno. La salida cruda **sin simplificar** —que incluye, para cada **ERROR**, la línea **DETAIL**: completa que el motor agrega automáticamente (fila específica que intentó insertarse, valores concretos en conflicto)—se archiva íntegra en **Anexos-Ejecutables-Entregable-06/log_04_pruebas_ejecucion.txt**:

N.º	Prueba	Restricción esperada	Resultado real del motor
1	Insertar una captura con un arreglo de 10 canales espectrales en vez de 11	chk_captura_espectro_11ch	ERROR: new row for relation "captura" violates check constraint "chk_captura_espectro_11ch"
2	Insertar un paciente con sexo = 'X'	chk_paciente_sexo	ERROR: new row for relation "paciente" violates check constraint "chk_paciente_sexo"

N.º	Prueba	Restricción esperada	Resultado real del motor
3	Insertar un segundo Consent con estado='vigente' para un paciente que ya tiene uno vigente	uq_consentimiento_vigente (RN-04)	ERROR: duplicate key value violates unique constraint "uq_consentimiento_vigente_por_paciente" - DETAIL: Key (paciente_id)=(...) already exists.
4	Insertar una captura referenciando un paciente_id inexistente	FK captura_paciente_id_fkey	ERROR: insert or update on table "captura" violates foreign key constraint "captura_paciente_id_fkey" - DETAIL: Key (paciente_id)=(...) is not present in table "paciente".
5	Eliminar un paciente que ya tiene un consentimiento asociado	FK consentimiento_paciente_id_fkey con ON DELETE RESTRICT	ERROR: update or delete on table "paciente" violates foreign key constraint "consentimiento_paciente_id_fkey" on table "consentimiento" - DETAIL: Key (id)=(...) is still referenced from table "consentimiento".
6	Insertar una segunda versión vigente (fecha_vigencia_hasta IS NULL) de tabla_correccion_altitud	uq_config_vigente (RN-02/CA-10)	ERROR: duplicate key value violates unique constraint "uq_config_vigente" - DETAIL: Key (tipo_parametro)=(tabla_correccion_altitud) already exists.
7	Insertar un AuditEvent con accion = 'destroy' (valor no admitido)	chk_auditevent_accion	ERROR: new row for relation "auditevent" violates check constraint "chk_auditevent_accion"
8 (control positivo)	Actualizar estado_cuenta de un usuario a 'inactivo' (operación válida)	Ninguna (debe tener éxito)	UPDATE 1 — confirmado con SELECT posterior

Las siete pruebas de restricción fallaron exactamente como se esperaba (ningún dato inválido quedó persistido) y el control positivo tuvo éxito, lo que descarta la hipótesis alternativa de que las restricciones estuvieran mal escritas y rechazaran *todo*, no solo los casos inválidos. Tras las pruebas, se ejecutó una limpieza que eliminó las dos filas huérfanas de **recurso_fhir** generadas por los intentos parcialmente exitosos de las pruebas 2 y 3 (el primer paso de cada una, la inserción en **recurso_fhir**, sí tuvo éxito antes de que el segundo paso fallara), devolviendo la

base de datos exactamente al estado del conjunto de datos ilustrativo original (verificado con `count(*)`): 34 `recurso_fhir`, 8 `paciente`, 9 `consentimiento`, idéntico a la Sección 2.4).

2.7 Nota de alcance de esta implementación

Se cierra esta sección reiterando, sin ambigüedad, cinco límites explícitos de lo aquí implementado, para que no se sobre-interprete su alcance:

1. **No es el servidor de producción.** Es un contenedor Docker efímero de validación, creado y destruido para este entregable. El aprovisionamiento del servidor real de la posta (EDT 2.2) es responsabilidad del Integrante 1 y no se ha ejecutado.
2. **El conjunto de datos es ficticio.** Los 8 pacientes, 6 cuidadores y demás filas del script DML son ilustrativos; no existe convenio institucional que permita cargar datos reales, y no se cargarían aunque existiera sin la autorización y anonimización correspondientes.
3. **No incluye programación de base de datos.** Deliberadamente, ningún script de esta Parte 1 define funciones, disparadores (*triggers*) ni procedimientos almacenados — ni siquiera para reforzar RN-06 (que un `caso_confirmacion.profesional_usuario_id` tenga rol `personal_medico`), que en esta fase se documenta como responsabilidad de la capa de aplicación y como candidato explícito a un disparador `BEFORE INSERT` en la Sección 3 (Parte 2, Semestre 2). La única vista (`reporte_evolucion`) es una sentencia `SELECT` simple, sin lógica procedural.
4. **No se midió rendimiento a escala real.** El plan de ejecución (`EXPLAIN`) de las consultas de agregación sobre este conjunto de 8 pacientes usa `Seq Scan` (barrido secuencial) en lugar de los índices creados, una decisión **correcta** del optimizador de PostgreSQL dado el tamaño trivial de las tablas — no una falla del diseño de índices. Esta observación se retoma explícitamente en la Autoevaluación como la razón por la cual el criterio “Implementación y consultas SQL” no se autoevalúa como “Excelente” sin matices: la corrección de las consultas está demostrada; su eficiencia a la escala real de 150-300 (Fase 0) o miles de registros (Fase 1/2) queda pendiente de una prueba de carga que no corresponde a esta etapa.
5. **La ejecución real es reproducible y verificable, pero fue realizada una sola vez, en una fecha puntual.** La evidencia de las Secciones 2.3 a 2.6 corresponde a una corrida concreta (2026-07-07T10:00:18Z UTC) contra un contenedor efímero ya destruido; no es un pipeline de integración continua que reejecute estas pruebas en cada cambio. Cualquiera puede repetirla siguiendo el procedimiento documentado en `Anexos-Ejecutables-Entregable-06/README.md` (mismos scripts, mismo digest de imagen), pero la reproducción continua y automatizada de esta verificación queda, como el resto de la programación avanzada de base de datos, fuera del alcance de esta Parte 1.

3. Consultas y Programación en Base de Datos

Fase Semestre 2 — pendiente. Esta sección se deja como *placeholder* de alcance, siguiendo instrucción explícita del encargo de este entregable: la competencia CE022 divide su evidencia en CE0221/CE0222 (Parte 1, desarrollada íntegramente en las Secciones 1 y 2) y CE0223/CE0224 (Parte 2, Semestre 2). No se desarrolla contenido evaluable aquí; se deja constancia únicamente de lo que la Parte 2 cubrirá, para que el alcance quede documentado desde ahora y no se improvise al inicio del Semestre 2:

- **Funciones y disparadores (*triggers*):** en particular, el disparador `BEFORE INSERT` sobre `clinico.caso_confirmacion` que verifique que `profesional_usuario_id`

corresponde a un usuario con `rol_id = personal_medico` (RN-06), ya anotado como pendiente en la Sección 2.7 y en el diccionario de datos (Sección 1.7); y un posible disparador sobre `clinico.determinacion` que valide la coherencia entre `severidad` y `valor_hb_final` contra la versión vigente de `config.configuracion_clinica` en el momento de la inserción (sin recalcular retroactivamente registros históricos, preservando la decisión de diseño ya justificada en la Sección 1.6).

- **Procedimientos almacenados** para las operaciones compuestas que hoy exigen varias sentencias coordinadas desde la aplicación (p. ej., la inserción atómica de `captura + correccion_altitud + determinacion + provenance + auditevent` que actualmente se demuestra mediante una transacción de cliente simple, Sección 2.4).
- **Consultas avanzadas** de mayor complejidad analítica: funciones de ventana (*window functions*) para calcular tendencias longitudinales de hemoglobina directamente en SQL (hoy delegado a la vista simple `reporte_evolucion` y, presumiblemente, a la capa de analítica en Python/PySAL del backend), y consultas recursivas si el modelo de seguimiento mensual lo requiere.
- **Control transaccional avanzado:** niveles de aislamiento explícitos, manejo de bloqueos (*locking*) para escrituras concurrentes desde múltiples nodos sincronizando simultáneamente (relevante para el patrón *outbox* del ADR-01, Entregable N.º 5), y la implementación real de la deduplicación por UUID idempotente ya exigida por dicho ADR.
- **Prueba de rendimiento a escala real**, con un volumen sintético de al menos 300 pacientes (Fase 0) y, si el tiempo lo permite, una proyección a los volúmenes de Fase 1/2, para verificar con EXPLAIN ANALYZE que los 60 índices ya creados en esta Parte 1 sí mejoran los planes de ejecución a esa escala (limitación explícita ya declarada en la Sección 2.7).

4. Seguridad y Administración de Base de Datos

Fase Semestre 2 — pendiente. Al igual que la Sección 3, esta sección se deja como *placeholder* de alcance (evidencia CE0224). Se deja constancia de tres precisiones importantes para evitar tanto la omisión como la duplicación con otros entregables ya existentes del proyecto:

- **No se duplica lo ya comprometido por Infraestructura Tecnológica.** El Entregable N.º 8 (Planificación de Seguridad, Área A) ya compromete, a nivel de infraestructura, cifrado en reposo AES-256 (o equivalente) para la base de datos PostgreSQL/PostGIS, copias de respaldo incremental diaria y completa semanal, y gestión de secretos (credenciales de base de datos) fuera del código fuente. Esta Sección 4, en su Parte 2, desarrollará específicamente los mecanismos **a nivel de motor de base de datos** que son complementarios y no redundantes con lo anterior: roles de PostgreSQL distintos de los roles de aplicación (`gestion.rol` es un catálogo de aplicación, no roles nativos de PostgreSQL con sus propios GRANT/REVOKE), políticas de *Row-Level Security* (RLS) para reforzar en el motor mismo restricciones como RN-08 (que ninguna consulta pueda leer `hexagono_cobertura` a nivel de individuo, porque estructuralmente no puede: no tiene FK a paciente) o para limitar el acceso de un usuario con rol `auditor` a solo lectura incluso si su credencial de aplicación fuera comprometida, y cifrado a nivel de columna (no solo de disco completo) para los campos más sensibles del diccionario de da-

tos (`paciente.nombres`, `paciente.apellidos`, `cuidador.nombre_completo`, `cuidador.telefono_contacto`).

- **Clasificación de sensibilidad de columnas**, insumo directo para decidir qué columnas requieren cifrado a nivel de columna: un ejercicio pendiente que debe cruzar el diccionario de datos completo de la Sección 1.7 de este documento contra la matriz de activos críticos ya iniciada en el Entregable N.º 8 (Planificación de Seguridad).
- **Auditoría a nivel de motor** (`pgAudit` o equivalente) como capa adicional —no sustituta— de la auditoría a nivel de aplicación que ya existe en el modelo de datos (`clinico.auditevent`, Sección 1.7): mientras `auditevent` registra *qué hizo la aplicación en nombre de un usuario*, una auditoría a nivel de motor registraría *qué sentencias SQL literales se ejecutaron*, una defensa en profundidad relevante si se llegara a sospechar una vulneración que evada la capa de aplicación.

Anexos

Evidencia externa y reproducibilidad de la ejecución real

Antes de los anexos propiamente dichos, se documenta aquí, de forma centralizada, el conjunto de **archivos externos al propio Markdown** que sustentan cada afirmación de “ejecución real” de la Sección 2. Esto responde directamente a una verificación adversarial de este entregable: que una afirmación de ejecución real, para ser creíble, no puede depender únicamente del texto narrativo de este mismo documento, sino que debe poder verificarse de forma independiente.

Todos los archivos siguientes viven en la carpeta hermana **Anexos-Ejecutables-Entregable-06/** (mismo nivel que este documento) y no dentro del Markdown:

Archivo	Contenido	Checksum SHA-256
<code>docker-compose.yml</code>	Definición exacta del contenedor efímero de validación	—
<code>01_ddl_hemoscan.sql</code>	(<code>postgis/postgis:16-3.4</code>) Script DDL completo (idéntico al resumido en el Anexo A)	<code>d101de476330b050d793f826e6fa45aafe931b1f6</code>
<code>02_dml_hemoscan.sql</code>	Script DML completo de carga ilustrativa (idéntico al resumido en el Anexo B)	<code>65e7f79bd0822260d3ed2cef90a6b5f0c0b261e01</code>
<code>03_consultas_verificacion.sql</code>	Las 10 consultas de verificación (idéntico al Anexo C)	<code>5fbf99528f0dd78b782ca0e42354fd0cd5d79b3d1</code>
<code>04_pruebas_integridad.sql</code>	Las 7 pruebas de violación + 1 control positivo (idéntico al Anexo D)	<code>b8b8a826e6083e988142db19b99df5fca92d862f9</code>
<code>log_01_ddl_ejecucion.txt</code> a <code>log_06_catalogo.txt</code>	Transcripciones crudas y sin editar de cada corrida real (DDL, DML, consultas, pruebas, conteo de catálogo, <code>\dt/\dv/\d</code>)	(ver <code>README.md</code> de esa carpeta)

Archivo	Contenido	Checksum SHA-256
README.md	Fecha exacta de la corrida (2026-07-07T10:00:18Z UTC), digest inmutable de la imagen Docker, procedimiento paso a paso para reproducir la verificación, y el detalle de las dos cifras que la ejecución real permitió corregir frente a lo que se había narrado (218->94 sentencias DDL)	—

Cualquier lector puede recalcular estos checksums (`sha256sum <archivo>` o `Get-FileHash <archivo> -Algorithm SHA256`) sobre los archivos de esa carpeta y compararlos contra esta tabla para confirmar que no fueron alterados después de la corrida documentada. Los Anexos A y B que siguen **no repiten el contenido íntegro** de sus respectivos scripts —ya archivado, íntegro, en los `.sql` externos citados arriba— sino su estructura y un extracto representativo, para no duplicar en este documento un contenido cuya fuente de verdad única y verificable ya son esos archivos.

Anexo A — Script DDL: estructura completa y extracto representativo (ejecutado sin errores contra PostgreSQL 16.4 + PostGIS 3.4.3; script íntegro en Anexos-Ejecutables-Entregable-06/01_ddl_hemoscan.sql)

```
-- =====
-- HemoScan Andino -- Modelo de datos relacional (PostgreSQL 16 + PostGIS 3.4)
-- Entregable N.6 (CE022 - Ingenieria de la Informacion) - Parte 1 (Semestre 1)
-- Script DDL completo. Convencion: snake_case, sin tildes/enies en identificadores.
-- Script INTEGRO (429 lineas) en Anexos-Ejecutables-Entregable-06/01_ddl_hemoscan.sql
-- checksum SHA-256: d101de476330b050d793f826e6fa45aafe931b1f6c86e7e737fff1e6cab10d50
-- =====

-- -----
-- 0. Extensiones y esquemas (namespaces por dominio, espejo de ADR-02)
-- -----

CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE SCHEMA IF NOT EXISTS gestion;
CREATE SCHEMA IF NOT EXISTS geo;
CREATE SCHEMA IF NOT EXISTS config;
CREATE SCHEMA IF NOT EXISTS clinico;

COMMENT ON SCHEMA gestion IS 'Usuarios, roles RBAC y agentes comunitarios de salud (soporte a
COMMENT ON SCHEMA geo IS 'Microrredes, establecimientos de salud (equivalentes a FHIR Local
COMMENT ON SCHEMA config IS 'Parametros clinicos versionados (modelo de inferencia, tabla de
COMMENT ON SCHEMA clinico IS 'Entidades clinicas alineadas a FHIR: Patient, Consent, Device,

-- =====
-- 1. ESQUEMA gestion (4 tablas: rol, usuario_sistema, agente_comunitario, alerta_stock)
-- =====

CREATE TABLE gestion.rol (
    id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre_rol VARCHAR(30) NOT NULL UNIQUE,
    descripcion VARCHAR(200) NOT NULL,
    permisos JSONB NOT NULL DEFAULT '[]'::jsonb,
    CONSTRAINT chk_rol_nombre CHECK (nombre_rol IN (
        'personal_cred', 'personal_medico', 'farmacia', 'jefatura_microred',
        'analista_diresa', 'administrador', 'auditor'
    ))
);
COMMENT ON TABLE gestion.rol IS 'Catalogo RBAC (RF-37, RNF-06). La fuente de verdad de autent

-- [... 3 tablas mas del esquema gestion (usuario_sistema, agente_comunitario,
--     alerta_stock), con sus indices y comentarios -- ver script integro ...]

-- =====
-- 2. ESQUEMA geo (3 tablas: microrred, establecimiento_salud, hexagono_cobertura)
-- =====

-- [... las 3 tablas del esquema geo, incluido el indice GIST sobre
--     establecimiento_salud.ubicacion -- ver script integro ...]

-- =====
-- 3. ESQUEMA config (2 tablas: modelo_inferencia, configuracion_clinica)
-- =====
```

difieren en calidad ni en nivel de detalle de los ya mostrados: cada una sigue el mismo patrón de PK, FK explícitas con su política **ON DELETE**, restricciones **CHECK** de dominio y **COMMENT ON** documentando su propósito y trazabilidad hacia el Entregable N.º 5, ya ilustrado en la Sección 1.5 (modelo lógico) y 1.7 (diccionario de datos completo, columna por columna, de las 24 tablas). El script `01_ddl_hemoscan.sql` es la única fuente íntegra y autorizada de este DDL; este extracto existe solo por legibilidad y no debe copiarse como referencia normativa si diverge de aquel.

Anexo B — Script DML: estructura completa y extracto representativo (carga de datos ilustrativos, ejecutado sin errores; script íntegro en Anexos-Ejecutables-Entregable checksum SHA-256 65e7f79bd0822260d3ed2cef90a6b5f0c0b261e01610dcfbfab0ae724fccc46b2)

```

-- =====
-- HemoScan Andino -- Script DML de carga (datos ILUSTRATIVOS/FICTICIOS)
-- Entregable N.6 (CE022) - Parte 1
-- ADVERTENCIA: Todos los datos de este script son FICTICIOS, generados
-- unicamente para validar el esquema. NO corresponden a ninguna nina, nino,
-- cuidador/a o profesional real. No existe convenio institucional vigente
-- con ninguna microred (Nota metodologica, Entregable N.5).
-- Script INTEGRO (348 lineas, 28 sentencias INSERT) en el .sql externo citado arriba.
-- =====

BEGIN;

-- -----
-- gestion.rol (7 filas -- catalogo RBAC completo)
-- -----
INSERT INTO gestion.rol (nombre_rol, descripcion, permisos) VALUES
('personal_cred',      'Personal de Crecimiento y Desarrollo que opera el dispositivo en campo',
'personal_medico',     'Profesional que confirma o descarta el diagnostico y prescribe tratamiento',
-- [... 5 filas mas de gestion.rol (farmacia, jefatura_microred, analista_diresa,
--      administrador, auditor) -- ver script integro ...]

-- -----
-- geo.microred (1 fila) y geo.establecimiento_salud (7 filas: 1 cabecera + 6 puestos)
-- -----
INSERT INTO geo.microred (id, nombre, red_salud, diresa_geresa, ubigeo_departamento, ubigeo_provincia,
('10000000-0000-0000-0000-000000000001',
'Microred de Salud Altiplano-Puno (caso ilustrativo y representativo)',
'Red de Salud San Roman (referencial, tipica de la zona)',
'DIRESA Puno (referencial)',
'21', '2111');
-- NOTA: ubigeo_provincia es ILUSTRATIVO. [DATO PENDIENTE DE VALIDAR: UBIGEO
-- exacto una vez exista convenio con una microred real].

INSERT INTO geo.establecimiento_salud
(id, microred_id, nombre, categoria, ubigeo, ubicacion, altitud_referencia_msnm, es_cabecera,
('20000000-0000-0000-0000-000000000001', '10000000-0000-0000-0000-000000000001',
'Centro de Salud I-4 "Altiplano" (cabecera de microred)', 'I-4', '210101',
ST_SetSRID(ST_MakePoint(-70.1181, -15.5000), 4326)::geography, 3824.0, TRUE);
-- [... 6 filas mas (Puestos de Salud "Comunidad A" a "F") -- ver script integro ...]

-- [... gestion.usuario_sistema (9 filas), gestion.agente_comunitario (2 filas),
--      gestion.alerta_stock (1 fila), config.modelo_inferencia (2 filas) y
--      config.configuracion_clinica (3 filas) -- ver script integro ...]

-- -----
-- clinico.correccion_altitud: fila del "Nino A", identica al wireframe P09/P13
-- del Entregable N.5 (altitud 3,812 m; version v1.2; LEVE)
-- -----
INSERT INTO clinico.correccion_altitud
(id, captura_id, configuracion_clinica_id, altitud_mde, hb_observada, hb_corregida, formu
('55000000-0000-0000-0000-000000000001', '54000000-0000-0000-0000-000000000001',
'41000000-0000-0000-0000-000000000002', 3812.0, 12.5, 10.8, 'v1.2', '2026-07-07T09:14:30-05
-- [... 5 filas mas de correccion_altitud (Nino B, D, E, F, G) -- ver script integro ...]

```

es enteramente ficticio, y las filas omitidas aquí (marcadas [...]) no difieren en naturaleza de las mostradas — el archivo `02_dml_hemoscan.sql` es la única fuente íntegra y verificable por checksum de este script; el detalle fila por fila ya se describe narrativamente en la Sección 2.4 (conteo exacto de filas por tabla, estados del ciclo de vida cubiertos, y el caso especial del ciclo revocar/re-otorgar del “Niño C”).

Anexo C — Script completo de consultas de verificación (10 consultas, todas ejecutadas; idéntico a Anexos-Ejecutables-Entregable-06/03_consultas_verificacion.sql, checksum SHA-256 5fbf99528f0dd78b782ca0e42354fd0cd5d79b3d1a4b0e790a02873e19dc2458; salida cruda completa de las 10 consultas en log_03_consultas_ejecucion.txt)

```
-- =====
-- Consultas de verificacion / evidencia (SELECT basico, sin logica procedural)
-- =====

-- 1. Conteo de filas por tabla
SELECT 'gestion.rol' AS tabla, count(*) FROM gestion.rol
UNION ALL SELECT 'gestion.usuario_sistema', count(*) FROM gestion.usuario_sistema
UNION ALL SELECT 'gestion.agente_comunitario', count(*) FROM gestion.agente_comunitario
UNION ALL SELECT 'gestion.alerta_stock', count(*) FROM gestion.alerta_stock
UNION ALL SELECT 'geo.microrred', count(*) FROM geo.microrred
UNION ALL SELECT 'geo.establecimiento_salud', count(*) FROM geo.establecimiento_salud
UNION ALL SELECT 'geo.hexagono_cobertura', count(*) FROM geo.hexagono_cobertura
UNION ALL SELECT 'config.modelo_inferencia', count(*) FROM config.modelo_inferencia
UNION ALL SELECT 'config.configuracion_clinica', count(*) FROM config.configuracion_clinica
UNION ALL SELECT 'clinico.recurso_fhir', count(*) FROM clinico.recurso_fhir
UNION ALL SELECT 'clinico.paciente', count(*) FROM clinico.paciente
UNION ALL SELECT 'clinico.cuidador', count(*) FROM clinico.cuidador
UNION ALL SELECT 'clinico.paciente_cuidador', count(*) FROM clinico.paciente_cuidador
UNION ALL SELECT 'clinico.consentimiento', count(*) FROM clinico.consentimiento
UNION ALL SELECT 'clinico.dispositivo', count(*) FROM clinico.dispositivo
UNION ALL SELECT 'clinico.captura', count(*) FROM clinico.captura
UNION ALL SELECT 'clinico.correccion_altitud', count(*) FROM clinico.correccion_altitud
UNION ALL SELECT 'clinico.determinacion', count(*) FROM clinico.determinacion
UNION ALL SELECT 'clinico.alerta_derivacion', count(*) FROM clinico.alerta_derivacion
UNION ALL SELECT 'clinico.caso_confirmacion', count(*) FROM clinico.caso_confirmacion
UNION ALL SELECT 'clinico.prescripcion', count(*) FROM clinico.prescripcion
UNION ALL SELECT 'clinico.provenance', count(*) FROM clinico.provenance
UNION ALL SELECT 'clinico.auditevent', count(*) FROM clinico.auditevent
UNION ALL SELECT 'clinico.control_mensual', count(*) FROM clinico.control_mensual
ORDER BY 1;

-- 2. Vista reporte_evolucion (JOIN de 3 tablas)
SELECT paciente_id, determinacion_id, hb_observada, hb_corregida, altitud_mde, severidad, estado
FROM clinico.reporte_evolucion;

-- 3. Consulta multi-tabla: agenda clinica completa por nino/a (5 JOIN, con LEFT JOIN
-- para incluir tambien a los ninos/as sin determinacion todavia)
SELECT
    p.nombres || ' ' || p.apellidos AS paciente,
    es.nombre AS establecimiento,
    d.severidad,
    d.estado,
    ca.hb_observada,
    ca.hb_corregida,
    ca.altitud_mde,
    ad.estado AS estado_alerta
FROM clinico.paciente p
JOIN geo.establecimiento_salud es ON es.id = p.establecimiento_asignado_id
LEFT JOIN clinico.determinacion d ON d.paciente_id = p.id
LEFT JOIN clinico.correccion_altitud ca ON ca.id = d.correccion_altitud_id
LEFT JOIN clinico.alerta_derivacion ad ON ad.determinacion_id = d.id
ORDER BY p.nombres;
```

tación reconocida en la Sección 2.7 (idéntica a la archivada en `log_03_consultas_ejecucion.txt`):

QUERY PLAN

```
-----  
HashAggregate  
  Group Key: es.nombre  
    -> Hash Join  
      Hash Cond: (p.establishment_assigned_id = es.id)  
        -> Seq Scan on paciente p  
        -> Hash  
          -> Seq Scan on establecimiento_salud es  
(7 rows)
```

El optimizador de PostgreSQL eligió correctamente un `Seq Scan` (barrido secuencial) sobre ambas tablas en lugar de usar el índice `ix_paciente_establecimiento` ya creado, porque con solo 7-8 filas por tabla un barrido secuencial es, en efecto, más rápido que la sobrecarga de leer un índice — el comportamiento **correcto y esperado** del optimizador a esta escala, no una falla del diseño de índices. La misma consulta con centenas o miles de filas (Fase 0 real: 150-300 niños; Fase 1/2: varios miles) sí utilizaría el índice, pero esa verificación queda para la Parte 2 (Sección 2.7).

Anexo D — Script completo de pruebas de integridad (7 pruebas de violación + 1 control positivo = 8 pruebas, todas con el resultado esperado; idéntico a Anexos-Ejecutables-Entregable-06/04_pruebas_integridad.sql, checksum SHA-256 b8b8a826e6083e988142db19b99df5fca92d862f9fb40f9fc452c61f2cac0b82; salida cruda completa con el detalle DETAIL: de cada error en log_04_pruebas_ejecucion.txt)

```
-- =====
-- Pruebas de integridad: intentos de insercion INVALIDA que deben ser
-- RECHAZADOS por el motor. Cada prueba es independiente (autocommit por
-- sentencia); un fallo esperado no aborta las pruebas siguientes.
-- =====
```

```
-- PRUEBA 1: espectro AS7341 con 10 canales en vez de 11 (debe FALLAR)
```

```
INSERT INTO clinico.captura (paciente_id, dispositivo_id, imagen_conjuntiva_ref, espectro_as7
VALUES ('50000000-0000-0000-0000-000000000001', '53000000-0000-0000-0000-000000000001', 'x',
        ARRAY[1,2,3,4,5,6,7,8,9,10]::numeric(10,4)[], 'x',
        ST_SetSRID(ST_MakePoint(-70.1,-15.5),4326)::geography, 0.9);
-- Resultado real: ERROR: new row for relation "captura" violates check
-- constraint "chk_captura_espectro_11ch"
```

```
-- PRUEBA 2: sexo invalido "X" (debe FALLAR)
```

```
INSERT INTO clinico.recurso_fhir (id, tipo_recurso) VALUES ('99999999-0000-0000-0000-00000000
INSERT INTO clinico.paciente (id, nombres, apellidos, fecha_nacimiento, sexo, establecimiento
VALUES ('99999999-0000-0000-0000-000000000001', 'Test', 'Invalido', '2025-01-01', 'X', '20000000-0
-- Resultado real: ERROR: new row for relation "paciente" violates check
-- constraint "chk_paciente_sexo"
```

```
-- PRUEBA 3: segundo Consent "vigente" para un paciente que ya tiene uno vigente
-- (debe FALLAR, RN-04)
```

```
INSERT INTO clinico.recurso_fhir (id, tipo_recurso) VALUES ('99999999-0000-0000-0000-00000000
INSERT INTO clinico.consentimiento (id, paciente_id, cuidador_id, estado, alcance, firma_digi
VALUES ('99999999-0000-0000-0000-000000000002', '50000000-0000-0000-0000-000000000001', '510000
        'vigente', '{"x":"y"}', 'hasdemo');
-- Resultado real: ERROR: duplicate key value violates unique constraint
-- "uq_consentimiento_vigente_por_paciente" -- DETAIL: Key (paciente_id)=(...) already exists
```

```
-- PRUEBA 4: captura referenciando un paciente inexistente (debe FALLAR, FK)
```

```
INSERT INTO clinico.captura (paciente_id, dispositivo_id, imagen_conjuntiva_ref, espectro_as7
VALUES ('00000000-1111-2222-3333-444444444444', '53000000-0000-0000-0000-000000000001', 'x',
        ARRAY[1,2,3,4,5,6,7,8,9,10,11]::numeric(10,4)[], 'x',
        ST_SetSRID(ST_MakePoint(-70.1,-15.5),4326)::geography, 0.9);
-- Resultado real: ERROR: insert or update on table "captura" violates foreign
-- key constraint "captura_paciente_id_fkey"
```

```
-- PRUEBA 5: borrar un paciente que ya tiene un consentimiento asociado
-- (debe FALLAR, ON DELETE RESTRICT)
```

```
DELETE FROM clinico.paciente WHERE id = '50000000-0000-0000-0000-000000000001';
-- Resultado real: ERROR: update or delete on table "paciente" violates foreign
-- key constraint "consentimiento_paciente_id_fkey" on table "consentimiento"
```

```
-- PRUEBA 6: segunda version "vigente" de tabla_correccion_altitud
-- (debe FALLAR, indice unico parcial uq_config_vigente)
```

```
INSERT INTO config.configuracion_clinica (tipo_parametro, version, valor_versionado, fecha_v
VALUES ('tabla_correccion_altitud', 'v154-invalido', '{"x":"y"}', NULL);
-- Resultado real: ERROR: duplicate key value violates unique constraint
-- "uq_config_vigente" -- DETAIL: Key (tipo_parametro)=(tabla_correccion_altitud) already exists
-- (Nota de proceso: una primera version de esta prueba uso un valor de
```

Anexo E — Glosario de términos técnicos de este entregable

Término	Definición breve
1FN / 2FN / 3FN	Primera, Segunda y Tercera Forma Normal; reglas de diseño relacional verificadas en la Sección 1.6
CE022	Competencia “Ingeniería de la Información” del Área C, evaluada por este entregable
<i>Class Table Inheritance</i>	Patrón de diseño relacional en el que varias tablas “hijas” comparten la clave primaria de una tabla técnica común; usado aquí para resolver la referencia polimórfica de <code>AuditEvent</code> (Sección 1.4, nota C)
CHECK (restricción)	Restricción de dominio de PostgreSQL que valida una condición booleana sobre cada fila antes de aceptarla
Diccionario de datos	Documento que describe, columna por columna, cada tabla de una base de datos: tipo, nulidad, clave y significado (Sección 1.7)
DDL (<i>Data Definition Language</i>)	Subconjunto de SQL para definir estructuras (<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code>); Anexo A
DML (<i>Data Manipulation Language</i>)	Subconjunto de SQL para manipular datos (<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>SELECT</code>); Anexos B y C
geography (PostGIS)	Tipo de dato espacial de PostGIS que calcula distancias sobre la superficie curva de la Tierra (a diferencia de <code>geometry</code> , que asume un plano)
GIST (índice)	Tipo de índice de PostgreSQL (<i>Generalized Search Tree</i>) usado para acelerar consultas espaciales sobre columnas <code>geography/geometry</code>
Índice único parcial	Restricción <code>UNIQUE</code> de PostgreSQL aplicada solo a las filas que cumplen una condición <code>WHERE</code> ; usado dos veces en este modelo para materializar RN-04 y RN-02/CA-10
JSONB	Tipo de dato de PostgreSQL para almacenar documentos JSON de forma binaria e indexable; usado para los parámetros clínicos versionados y otros campos semi-estructurados
Modelo conceptual / lógico / normalizado	Los tres niveles de abstracción de un modelo de datos exigidos por la rúbrica de este entregable (Secciones 1.3, 1.5 y 1.6 respectivamente)
ON DELETE RESTRICT / CASCADE / SET NULL	Políticas de PostgreSQL ante el borrado de una fila referenciada por una llave foránea; este modelo usa predominantemente <code>RESTRICT</code> para datos clínicos (nunca borrado accidental en cascada)
psql	Cliente de línea de comandos oficial de PostgreSQL, usado para ejecutar los cuatro scripts de este entregable
Referencia polimórfica	Relación en la que una tabla puede apuntar a filas de <i>varias</i> tablas distintas según el valor de otra columna; problema típico de modelar el estándar FHIR (<code>AuditEvent.entity</code>) en SQL relacional (Sección 1.4, nota C)
Vista (VIEW)	Consulta <code>SELECT</code> almacenada con nombre propio, sin almacenamiento de datos independiente; usada para <code>clinico.reporte_evolucion</code>

Para los términos generales del proyecto (FHIR, HL7, LOINC, H3, PMTiles, RBAC, RF/RNF/RN, etc.), véase el Anexo A del Entregable N.º 5, no reproducido aquí para evitar duplicación.

Anexo F — Índice de diagramas de este entregable

N.º	Nombre	Sección	Tipo
1	Vista general de la arquitectura de información (4 esquemas)	1.2	Arquitectura de datos
2	Modelo E-R del núcleo clínico (esquema clínico)	1.3	Entidad-Relación
3	Resolución de la referencia polimórfica FHIR	1.3	Entidad-Relación
4	Modelo E-R del dominio geoespacial (esquema geo)	1.3	Entidad-Relación
5	Modelo E-R del dominio de configuración y gestión	1.3	Entidad-Relación

Nota sobre la numeración: siguiendo la misma convención ya establecida por los Entregables N.º 5, 7 y 8 (cada entregable reinicia su propia numeración de diagramas en 1), este documento no continúa la numeración de los 15 diagramas del Entregable N.º 5.

Anexo G — Trazabilidad hacia el Entregable N.º 5

Elemento del Entregable N.º 5	Cómo se materializa en este Entregable N.º 6
RF-02 (captura sincronizada de 11 canales AS7341)	<code>clinico.captura.espectro_as7341</code> <code>NUMERIC(10,4)[] +</code> <code>CHECK(array_length(...)=11)</code>
RF-07, RN-03 (nunca altitud cruda de GPS)	<code>geo.establecimiento_salud.altitud_referencia_msnm</code> y <code>clinico.correccion_altitud.altitud_mde</code> documentados explícitamente como derivados de MDE; <code>clinico.captura.coordenada_gps</code> documentado como “nunca se usa su altitud implícita”
RF-09, RN-02, CA-10 (corrección auditable y versionada)	<code>config.configuracion_clinica</code> (JSONB versionado) + índice único parcial <code>uq_config_vigente</code>
RF-12, RN-04 (bloqueo sin consentimiento vigente)	Índice único parcial <code>uq_consentimiento_vigente_por_paciente</code> ; verificado con salida real en la Sección 2.6, prueba 3
RF-28 (generación atómica de recursos FHIR)	Transacción <code>BEGIN...COMMIT</code> del script DML (Sección 2.4); programación transaccional avanzada diferida a la Sección 3
RF-31, RF-38, RN-11 (AuditEvent referencia sin excepción a cualquiera de los 5 recursos)	Tabla técnica <code>clinico.recurso_fhir</code> (Sección 1.4, nota C)
ADR-05 (autocalibración por altitud como módulo independiente y auditable)	<code>captura</code> , <code>correccion_altitud</code> y <code>determinacion</code> como tres tablas separadas con relación 1:1 forzada por <code>UNIQUE</code> , no una tabla ancha única

Elemento del Entregable N.º 5	Cómo se materializa en este Entregable N.º 6
ADR-06 (6 recursos FHIR fijos; <code>Location</code> no es uno de ellos)	Nota de coherencia explícita en la Sección 1.4 sobre <code>geo.establishment_salud</code>
ADR-10 (PostgreSQL + PostGIS como único motor)	Todo este entregable; verificado con <code>SELECT PostGIS_Full_Version()</code> real (Sección 2.1)
Diagrama 6 (20 clases de dominio)	Sección 1.4 (mapeo completo, clase por clase)
Diagrama 11 (estados de la Determinación/Observation)	<code>clinico.determinacion.estado</code> , dominio <code>CHECK</code> con los mismos 9 nombres de estado
Diagrama 12 (estados del Consentimiento)	<code>clinico.consentimiento.estado</code> + <code>fecha_revocacion</code> ; ciclo revocar/re-otorgar demostrado con datos reales del “Niño C” (Sección 2.5, consulta 9)
Sección 4.5 (mapeo a los 6 recursos FHIR)	Diccionario de datos completo (Sección 1.7), tabla por tabla
EDT 4.2 “Modelo de datos y mapeo FHIR” (Entregable N.º 3)	Este entregable satisface íntegramente su criterio de aceptación (“cada campo clínico mapeado a un recurso/atributo FHIR”)

Autoevaluación frente a la rúbrica

Nota sobre la escala usada. La rúbrica oficial de este entregable define únicamente **cuatro niveles cualitativos: Excelente, Bueno, Regular y Deficiente**, sin escala numérica asociada. La tabla siguiente usa exclusivamente esos cuatro niveles, sin introducir rangos numéricos ni categorías híbridas no contempladas en la rúbrica.

Criterio	Nivel alcanzado	Justificación
Modelado de datos	Excelente	El modelo cubre 24 tablas y 1 vista en 4 esquemas, con un diagrama entidad-relación completo (5 diagramas), una reconciliación explícita y honesta —clase por clase, incluyendo cuatro notas de reconciliación detalladas— con el Diagrama 6 del Entregable N.º 5, y una resolución documentada de dos problemas de modelado no triviales del estándar FHIR (referencia polimórfica de <code>AuditEvent</code> y cardinalidad 1:1 auditable de la cadena Captura-CorrecciónAltitud-Determinación). El límite honesto: los valores exactos de varios parámetros clínicos (NTS 213-MINSA/DGIESP-2024) siguen sin confirmarse, por lo que el modelo se valida estructuralmente pero no puede todavía validarse contra los valores clínicos reales que administrará.
Integridad y consistencia de datos	Excelente	Integridad referencial completa: 39 llaves foráneas reales (ninguna referencia polimórfica resuelta con una FK débil), 40 restricciones CHECK de dominio y 2 índices únicos parciales que materializan reglas de negocio (RN-04, RN-02/CA-10) directamente en el motor. A diferencia de una declaración de intención, esta integridad se demostró empíricamente y de forma independientemente verificable: 7 intentos deliberados de insertar datos inconsistentes fueron rechazados por el motor real (más 1 control positivo que confirmó que las restricciones no rechazan indiscriminadamente toda operación), y toda la corrida —scripts, <code>docker-compose.yml</code> y transcripciones crudas— queda archivada con checksum SHA-256 en Anexos-Ejecutables-Entregable-06/, no solo citada como texto dentro de este documento.

Criterio	Nivel alcanzado	Justificación
Implementación y consultas SQL	Bueno	Las consultas son correctas , verificadas con ejecución real y externamente reproducible: uniones de hasta 5 tablas, agregaciones con FILTER/HAVING , dos consultas geoespaciales PostGIS genuinas (ST_Distance , operador <->) y una traza de auditoría multi-tabla, todas contra una instancia real de PostgreSQL 16.4 + PostGIS 3.4.3. Se autoevalúa en “Bueno” y no en “Excelente” porque el criterio textual de la rúbrica para ese nivel exige consultas “correctas y eficientes”, y la eficiencia no se demostró a escala real: el propio EXPLAIN de la Sección 2.5/Anexo C muestra Seq Scan en lugar de uso de índices, correcto para 8 filas pero no demostrado para los volúmenes reales de Fase 0 (150-300) o Fase 1/2 (miles) — una prueba de carga pendiente que corresponde a la Parte 2 de este entregable en el Semestre 2 (Sección 2.7, límite 4).
Consultas avanzadas y programación en BD (Sección 3)	Fuera de alcance de esta Parte 1 — no es un nivel de la rúbrica	Deliberadamente fuera de alcance de esta Parte 1, por instrucción explícita del encargo; no se autoasigna ninguno de los cuatro niveles porque no hay desarrollo evaluable que calificar. Se documentó únicamente el plan de contenido (funciones, disparadores, procedimientos almacenados, control transaccional avanzado, prueba de carga) como <i>placeholder</i> de alcance para el Semestre 2.

Criterio	Nivel alcanzado	Justificación
Seguridad y administración de BD (Sección 4)	Fuera de alcance de esta Parte 1 — no es un nivel de la rúbrica	Ídem. Se documentó explícitamente la división de responsabilidad con el Entregable N.º 8 (Infraestructura Tecnológica), que ya compromete cifrado en reposo y respaldos a nivel de infraestructura, para que la Parte 2 desarrolle específicamente los mecanismos a nivel de motor de base de datos (roles nativos, RLS, cifrado de columna) sin duplicar ni contradecir lo ya comprometido.

Nota final sobre las brechas identificadas. La brecha más significativa de esta Parte 1 —la falta de una prueba de rendimiento a escala real— es, deliberadamente, la misma que se declaró como pendiente en la Sección 2.7 y que se traslada de forma explícita a la Parte 2 (Semestre 2), junto con la implementación real del disparador que reforzaría RN-06 a nivel de motor. A diferencia de las brechas señaladas en entregables anteriores del proyecto (que en su mayoría dependían de un convenio institucional externo, fuera del control del equipo), esta brecha específica **sí está enteramente bajo control del equipo** y se resolverá con una prueba de carga sintética, no con una gestión externa pendiente — una distinción que vale la pena remarcar en la verificación cruzada final del proyecto (tarea transversal ya identificada en el Entregable N.º 5). Una segunda brecha, ya reconocida en la nota metodológica inicial (punto Sexto), es de extensión: aun tras condensar los Anexos A y B a un extracto representativo y mover el contenido íntegro a archivos externos verificables, no se afirma que el documento quede ya dentro del rango de 15-25 páginas de la plantilla del producto; esa evaluación formal queda pendiente de la integración con la Parte 2.